Training anSwitch V7

USING & MANAGING THE CTI API

Classification: For Internal Use
Status: Released
Version: E3.2.1
Author: D. Bochsler

INTRODUCTION & MOTIVATION

This training covers the topics:

- Overview of the anSwitch V7 CTI API
- How to use the CTI API in a CTI-application/client
- Setting Up the CTI API Interaction with a customer CTI-application
- Brief description of the supported ECMA CSTA services

After this training, the trainee is enabled:

- To understand how the CTI API is working
- To understand how different use case can be realized
- To configure a customer PBX for interacting with a CTI-application via the CTI API

IT'S NOT MAGIC

IT'S "KNOW HOW"

TABLE OF CONTENTS

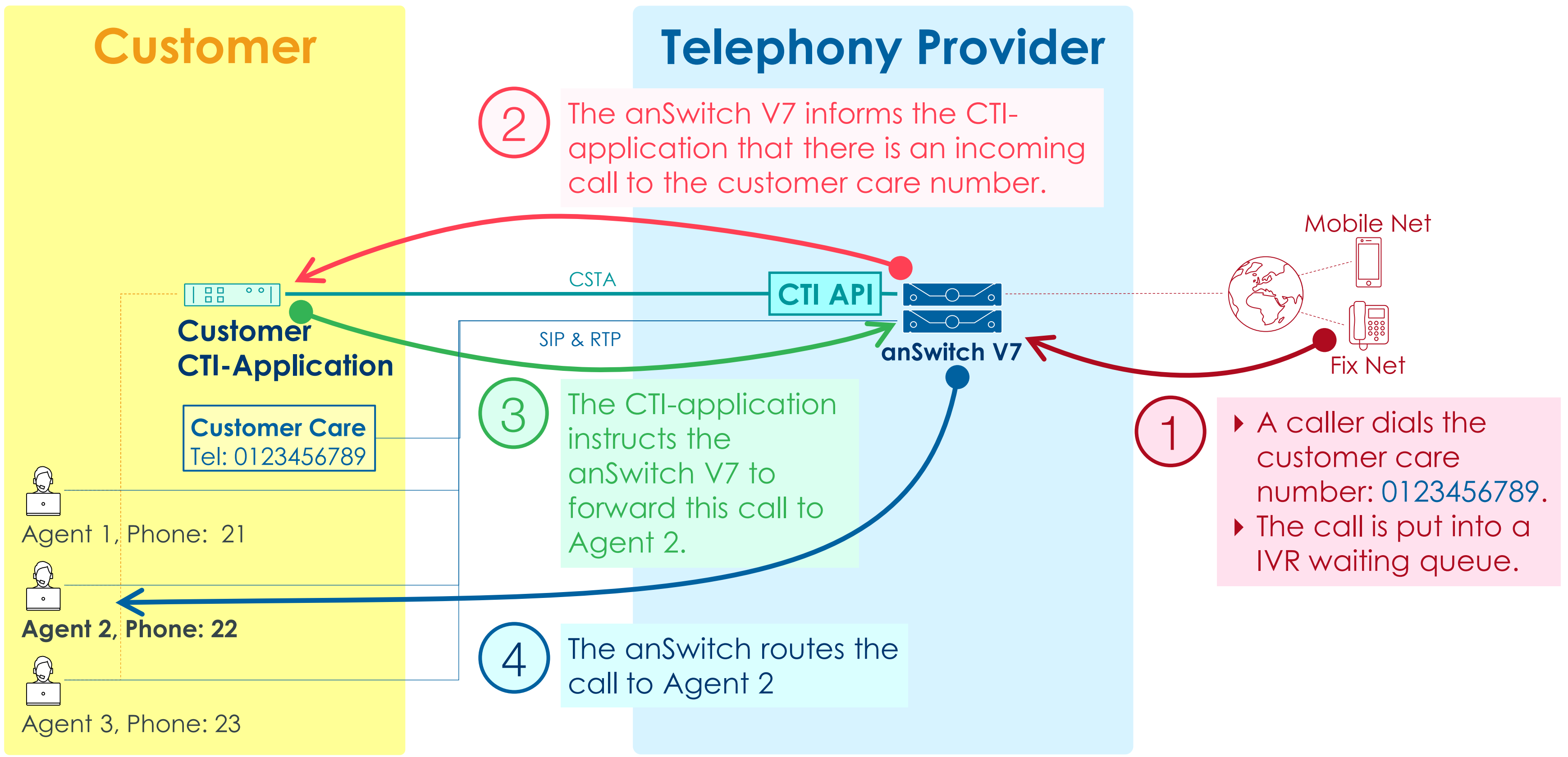
1 OVERVIEW OF THE CTI API

OVERVIEW OF THE CTI API

- The anSwitch V7 contains a CTI API which is based on the CSTA protocol.
- The CTI API offers customers the possibility to implement applications such as:
  - Call-Center applications — Example see Frontstage V5
  - anDesktop PC Application — Example see anSwitch V7 system phone "anDesktop"
  - Phone-like Web applications — Example see anSwitch V7 feature "Click-to-Call"
  - Remote control of SIP phones out of MS Teams — Example see anSwitch V7 feature "anCall"
  - Proprietary Web phones
  - etc.

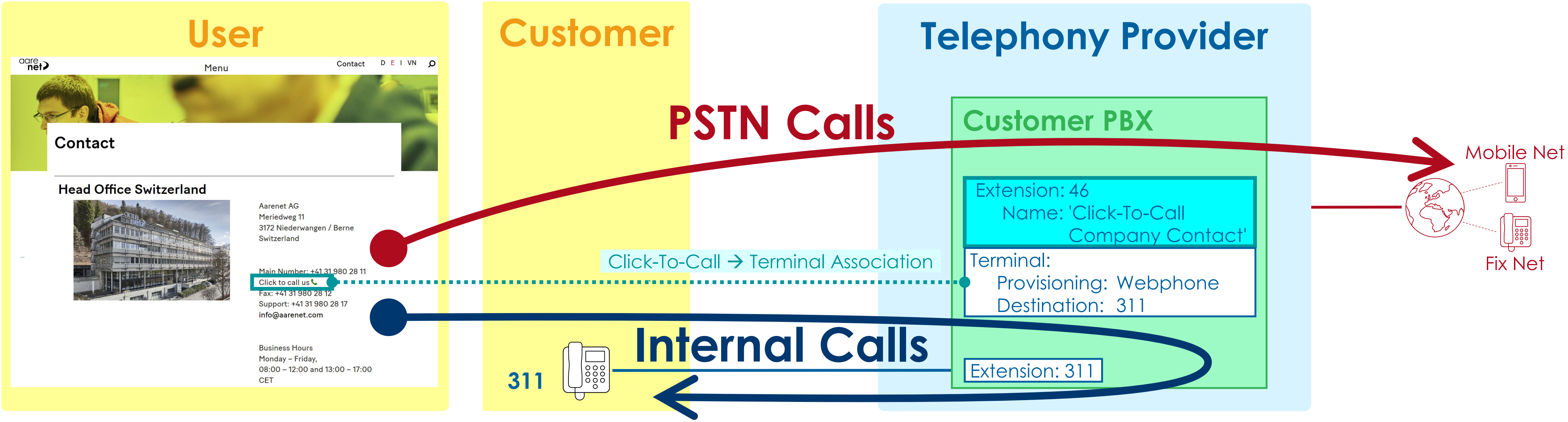
USE CASE OF THE PURE CSTA CTI API

- The pure CSTA CTI API enables the implementation of CTI-application, e.g. call center.



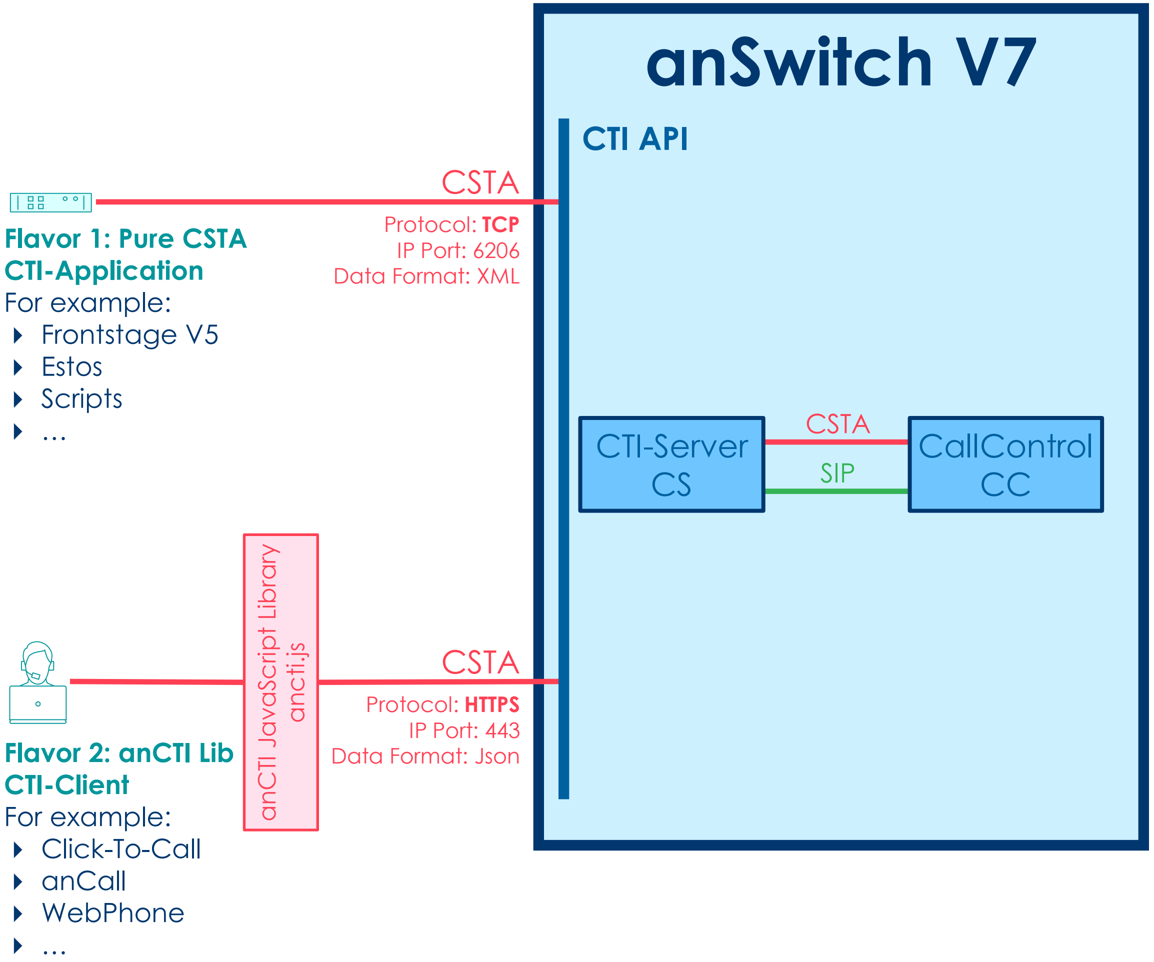
USE CASE OF THE ANCTI JAVASCRIPT LIBRARY

- The anCTI JavaScript library enables the implementation of Web based applications, e.g.:
  - Proprietary customer Webphone
  - Click-to-Call button in a Web page



OVERVIEW OF THE CTI API FLAVORS

- The CTI API is usable in two flavors:
  - Flavor 1: Pure CSTA
    - The CTI-client and CTI-Server exchange CSTA messages in CSTA standard XML format.
    - For the message transfer TCP is used.
    - The CTI-client application is fully responsible for the correct interworking.
    - ➔ *This flavor is the choice of the CSTA nerds.*
  - Flavor 2: anCTI JavaScript Library
    - The anCTI JavaScript library hides completely the methods of CSTA.
    - As transport protocol HTTPS is used.
    - The CTI-client Web application uses methods for call and device handling.
    - It handles emitted events from the CTI-Server.
    - Proprietary methods extend the CSTA standard for the developer's convenience.
    - ➔ *This flavor is the choice of Web designers and Web application developers.*



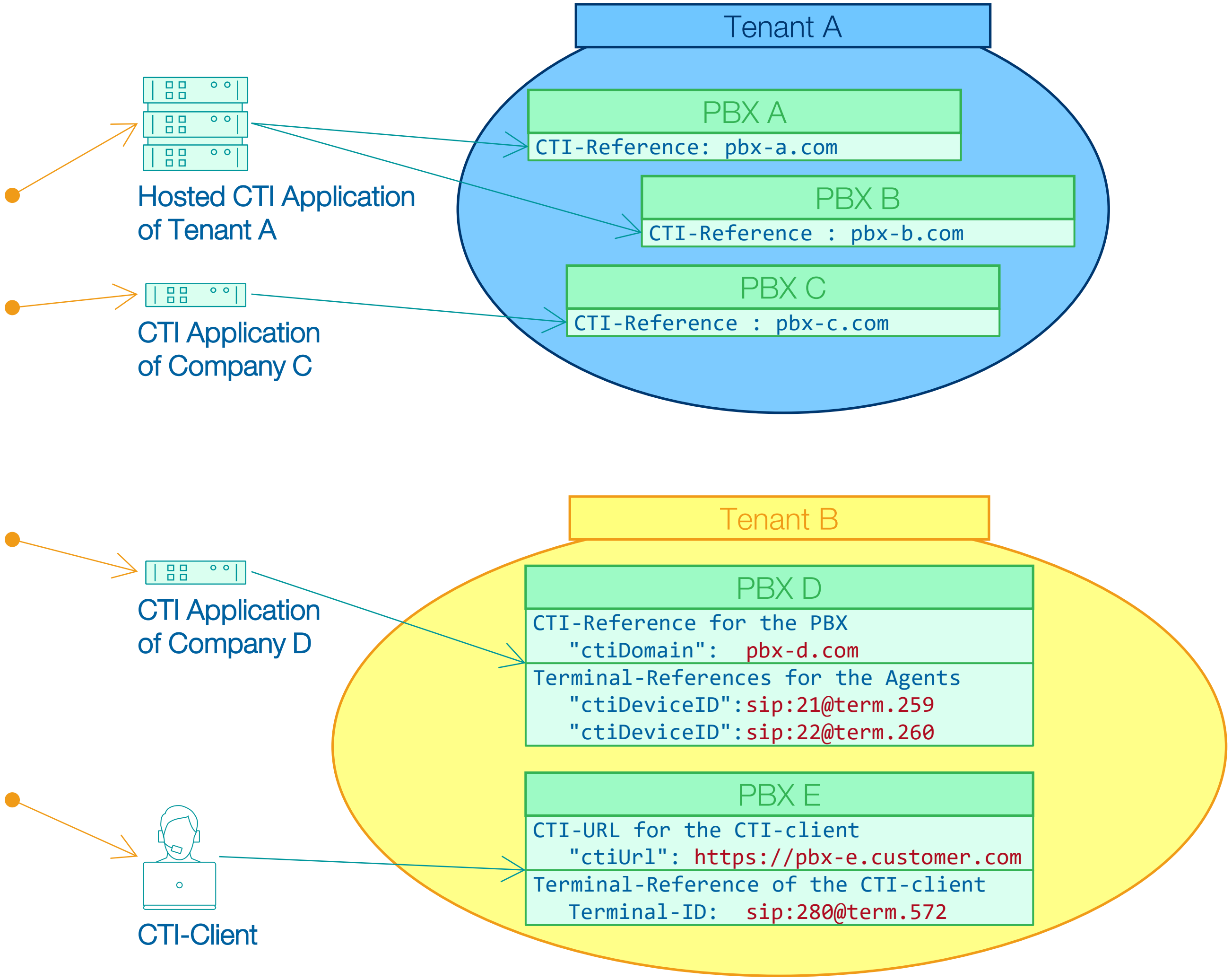
OVERVIEW OF THE CTI API FUNCTIONALITIES

- The CTI API provides functionalities as:
  - Handling incoming and outgoing calls to and from agents
  - Call hold, transfer, pick-up, conference
  - Call reject, do not disturb
  - Handling VoiceMail Box messages
  - Call recording
  - Supervising connections with barge in, whisper, listen
  - Receiving and sending DTMF
  - Device monitoring, snapshot devices of a calls and presence of a device
  - etc.

- Additional proprietary functionalities provided by the anCTI JavaScript library:
  - Handling of audio and video streams

OVERVIEW OF THE CTI API REFERENCING SYSTEM

- The CTI API referencing method for PBXs and terminals allow:
  - Single or shared CTI-application deployment:
    - anSwitch V7 provider or tenant can offer hosted CTI services.
    - Customers can have their own CTI-application.
- Resource referencing:
  - Pure CSTA:
    - A unique CTI-reference "ctiDomain" separates the different customer PBXs in the CTI-Server component.
    - The resources, e.g. phones, a CTI-application can monitor and manage must be declared with unique Terminal-references.
  - anCTI JavaScript Library:
    - A CTI URL "ctiDomain" is the peering URL for the CTI-clients, e.g. Web-phone.
    - The CTI-client is represented with a unique terminal-ID.



OVERVIEW OF THE CTI API SUPPORTED STANDARDS: CSTA & WEBRTC

- Call signaling and events are handled according the ECMA **CSTA** standard
  - The CTI API is based on the standardized ECMA CSTA protocol defined in ECMA-269 / ECMA-323.
- The media stream management of the anCTI JavaScript library is handled according **WebRTC**
  - Audio and video connections are possible.
  - Audio and video input and output interfaces are selectable.

* CSTA stands for: Computer Supported Telecommunications Applications
(For further introductory information to computer telephony integration CTI, see this Wikipedia article)

OVERVIEW OF SUPPORTED CSTA SERVICES & EVENTS

- The ECMA CSTA in the current Phase III defines 136 services.
- The CTI API of the anSwitch V7 covers a subset of the CSTA services.
- Currently the anSwitch V7 supports **38** CSTA services & events
- The supported CSTA services allow the full control of all anSwitch V7 features.

OVERVIEW OF SUPPORTED CSTA SERVICES & EVENTS

CSTA Services supported:

Device Service

- MakeCall
- JoinCall
- MonitorStart
- MonitorStop
- DirectedPickupCall
- SetForwarding
- GetForwarding
- SetDoNotDisturb
- GetDoNotDisturb
- SnapshotDevice
- SnapshotCall
- GetPresenceState

Call Services

- AnswerCall
- ClearConnection
- SingleStepTransferCall
- HoldCall
- RetrieveCall
- GenerateDigits
- DeflectCall
- DirectedPickupCall
- TransferCall
- ConferenceCall
- PlayMessage
- RecordMessage
- Stop

CSTA Events supported:

Device Events

- PresenceStateEvent

Call Events

- ServiceInitiatedEvent
- FailedEvent
- ConnectionClearedEvent
- OriginatedEvent
- DeliveredEvent
- DivertedEvent
- EstablishedEvent
- HeldEvent
- RetrievedEvent
- TransferredEvent
- DtmfDetectedEvent
- StopEvent

OVERVIEW OF ADDITIONALLY SUPPORTED SERVICES & EVENTS BY THE ANCTI JAVASCRIPT LIBRARY

- anCTI JavaScript Library Services supported:

  Agent Services
  - StartApplicationSession
  - StopApplicationSession
  - GetDevices
  - GetDevice
  - SetAudioDeviceId
  - SetVideoDeviceId
  - On

  Device Service
  - GetCall
  - GetCalls

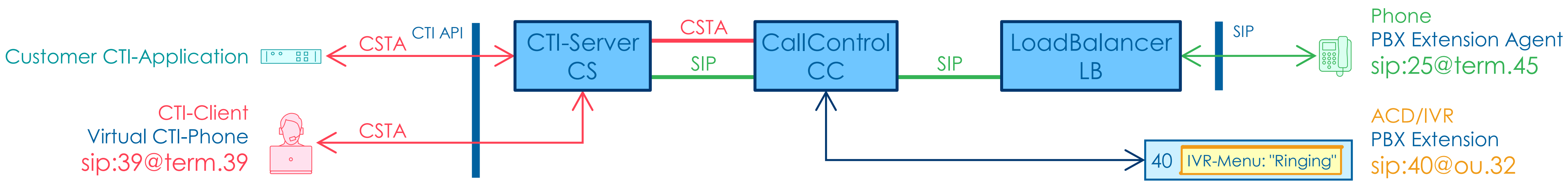
  Call Services
  - UpdateCall

- anCTI JavaScript Library Events supported:

  Agent Events
  - ApplicationSessionStartedEvent
  - ApplicationSessionTerminatedEvent
  - LocalStreamEvent
  - RemoteStreamEvent

OVERVIEW OF THE CTI API ARCHITECTURE

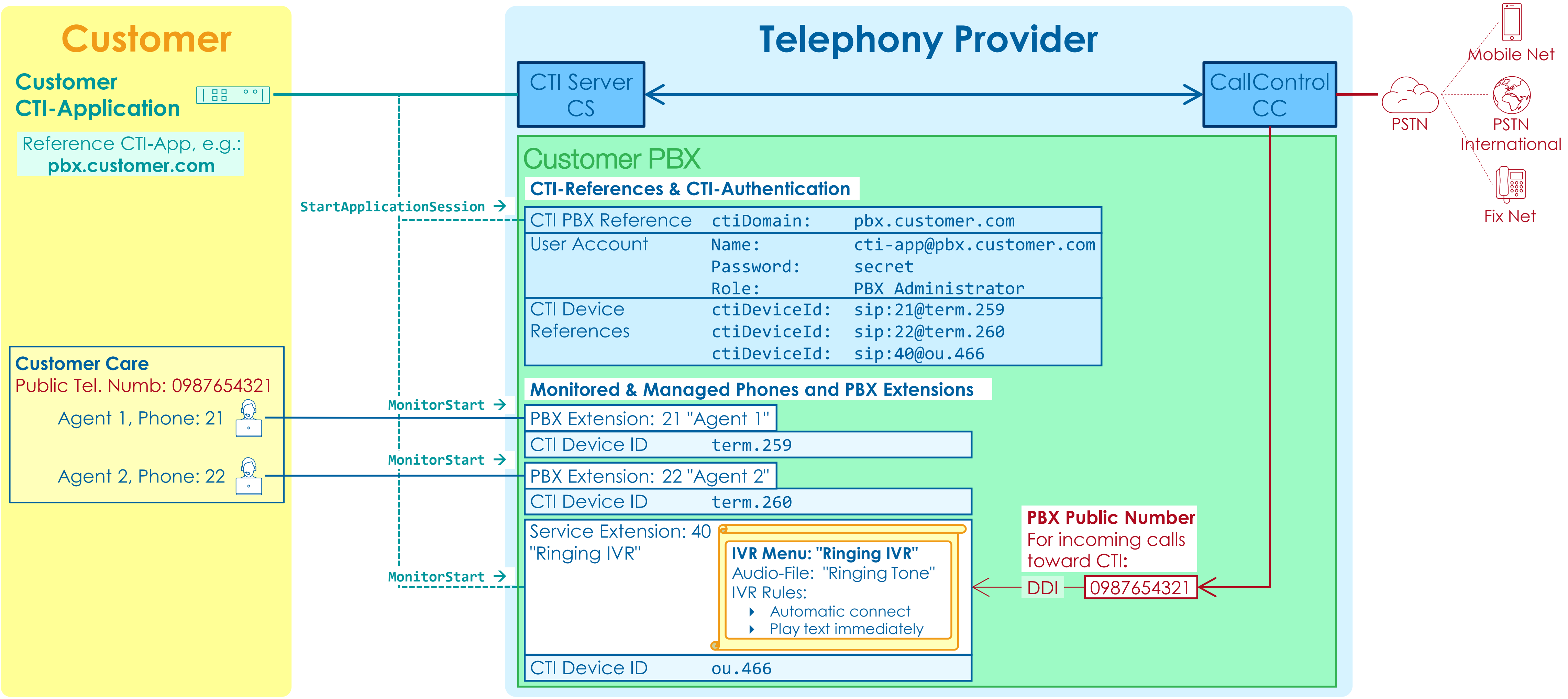
- The CTI-Server CS component of the anSwitch:
  - Acts as a gatekeeper for CSTA services.
  - Performs authentication and authorization and forwards the CSTA messages to the CallControl.
  - Multiplexes the events that it receives from the CallControl for given devices and dispatches them to all monitoring applications.



2 PBX CONFIGURATION FOR INTERACTING WITH A CTI-APPLICATION

OVERVIEW INTERWORKING CTI-APPLICATION WITH THE PBX

- This type of interworking with the PBX uses the pure CSTA.



OVERVIEW OF CUSTOMER PBX CONFIGURATION

- Overview of the needed definitions and configuration steps:
  1. Define the mutual definitions for interworking.

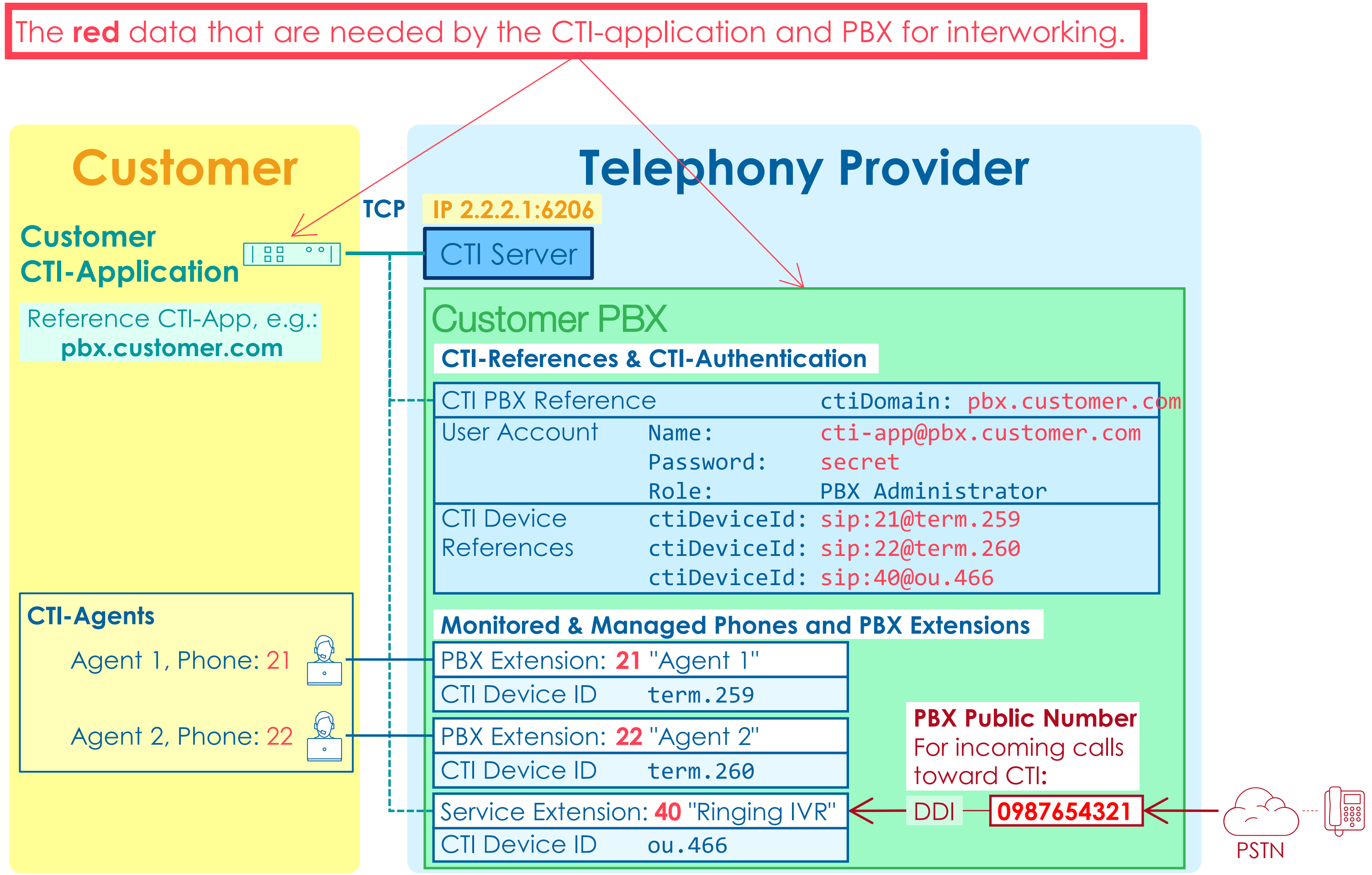
  2. Create the PBX or extend an existing PBX and assign the agent phones.
  3. Configure the CTI user account with role PBX Administrator.
  4. Configure the defined mutual CTI PBX-reference.
  5. Configure the CTI device references.

  6. Configure optional features useful for a CTI-application.
     - Call recording
     - Supervisor barge-in/whisper/listen

  7. Check and adjust the IP network & security aspects.

1. DEFINE THE MUTUAL DEFINITIONS FOR INTERWORKING

- Define the data the CTI-application needs for interworking:
  - ☐ IP address and port of the anSwitch V7 CTI-Server
  - ☐ Unique CTI PBX reference for "ctiDomain"
  - ☐ User authentication username and password.
  - ☐ CTI Public number for incoming calls.
  - ☐ All agent phone numbers
  - ☐ All phone numbers of needed service Extensions for the CTI-application.
  - ☐ When the agent phone are assigned all CTI device references "ctiDeviceId".



2. CREATE THE PBX OR EXTEND AN EXISTING PBX

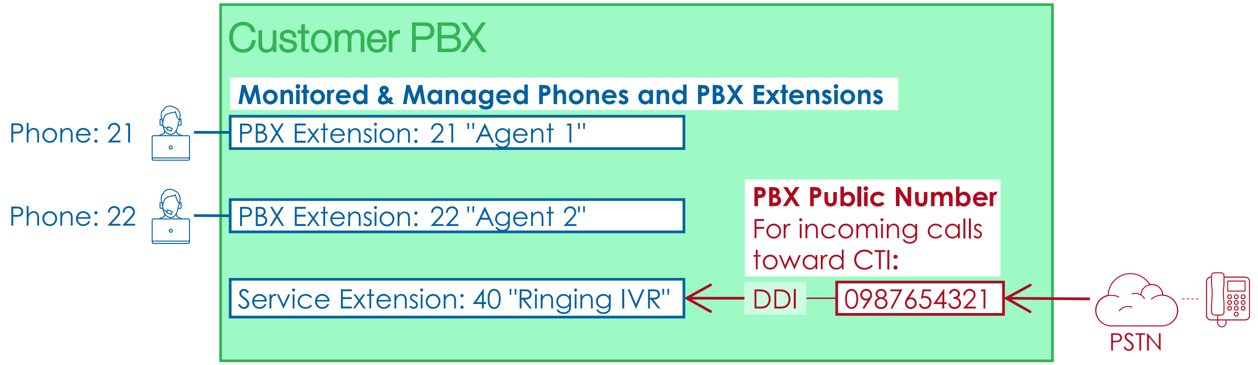
- Create the PBX or extend an existing PBX.
  a. If needed create a new PBX.

  b. Create the needed PBX Extensions with the defined internal number.
  c. Create the needed service Extensions with the defined internal number.
     a. Configure the needed IVR behavior.
     b. Configure a fallback to the agent phones.

  d. Configure the DDI for incoming calls toward the CTI service extension with the defined public number.

  e. Assign the agent phone to their extensions.
  f. Configure the phone and register them.

  g. For testing the agent phones do incoming and outgoing calls.



3. CONFIGURE THE CTI USER ACCOUNT WITH ROLE PBX ADMINISTRATOR

- Configure the CTI user account with role PBX Administrator.

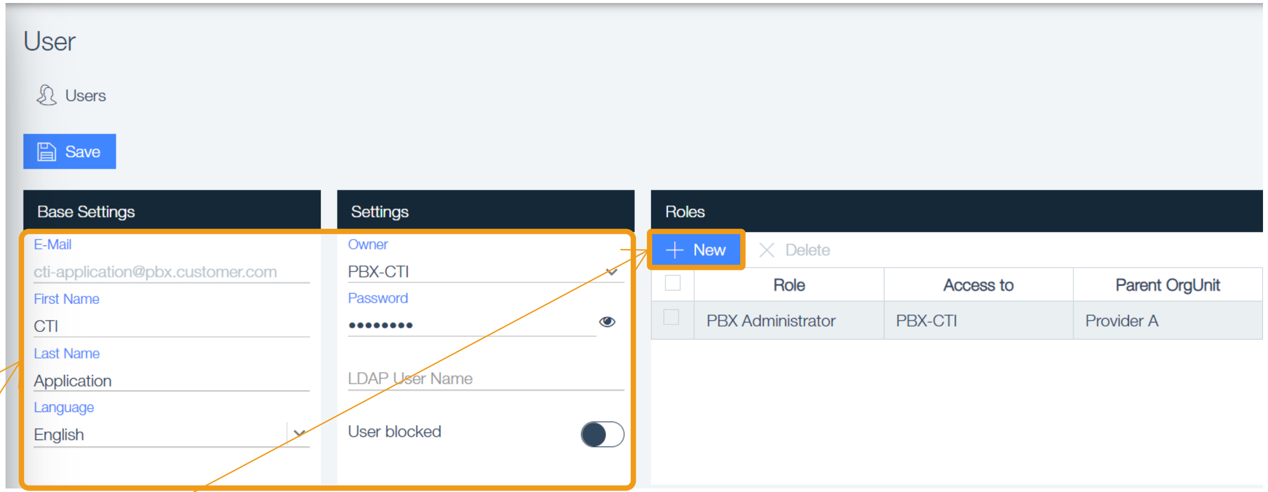
Name	Value	Remark / Example
E-mail	………..	Username, e.g.: cti-app@pbx.customer.com
Password	………..	
Role	PBX Administrator	
Access to OrgUnit	………..	PBX to which the CTI-application is assigned to

Customer PBX

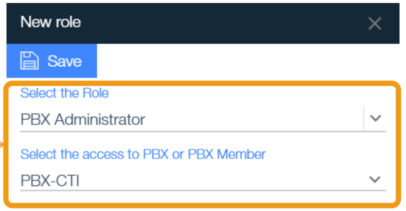
CTI-References & CTI-Authentication

User Account	Name:	cti-app@pbx.customer.com
	Password:	secret
	Role:	PBX Administrator

- Configure a user account for the CTI-application of the desired PBX:
  > Menu: Operator
    > Sub-Menu: Users
      > Add new CTI user with its login credentials



- Configure the role for accessing the PBX:
  > At tile "Roles" click button: + New
  > Assign the access to the PBX:
    - A suitable role e.g.: PBX Administrator
    - Select the PBX



4. CONFIGURE THE DEFINED MUTUAL CTI PBX REFERENCE

- Configure the defined mutual CTI PBX-reference.

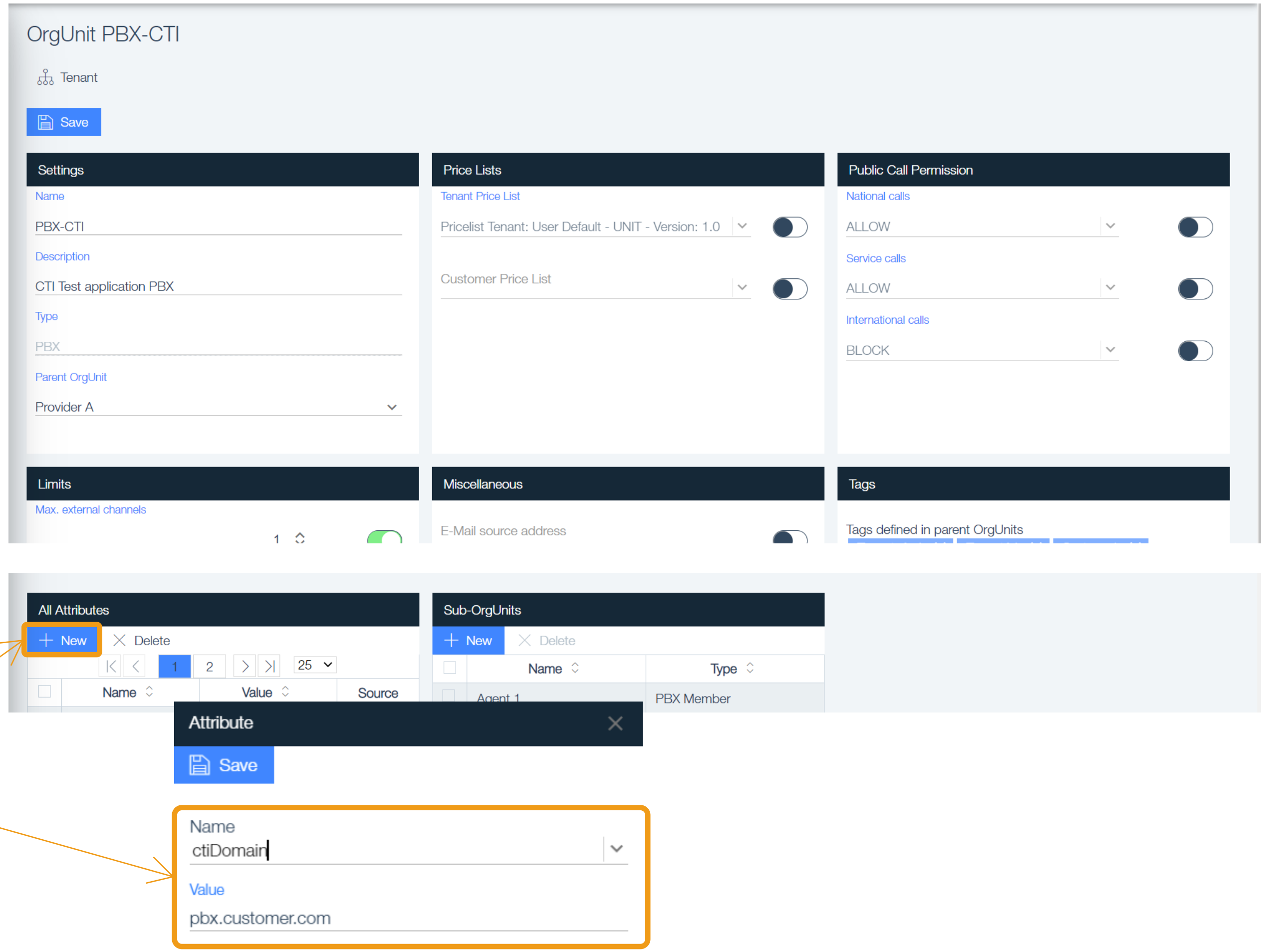
Name	Value	Remark / Example
`ctiDomain`	....	PBX attribute, e.g.: `pbx.customer.com`

Customer PBX

CTI-References & CTI-Authentication

CTI PBX Reference `ctiDomain: pbx.customer.com`

- Configure the customer domain name as attribute ctiDomain of the PBX OrgUnit:
  - > Menu: Operator
    - > Sub-Menu: Organization Units
      - > Navigate to the PBX OrgUnit
        - > Add in tile "Attributes a new attribute click: + New
          - > Configure the attribute:
            - Name: ctiDomain
            - Value: `CUSTOMER_CTI_DOMAIN`



5. CONFIGURE THE CTI DEVICE REFERENCES

A. Obtain the PBX CTI Device ID for the managed service extensions.

Customer PBX

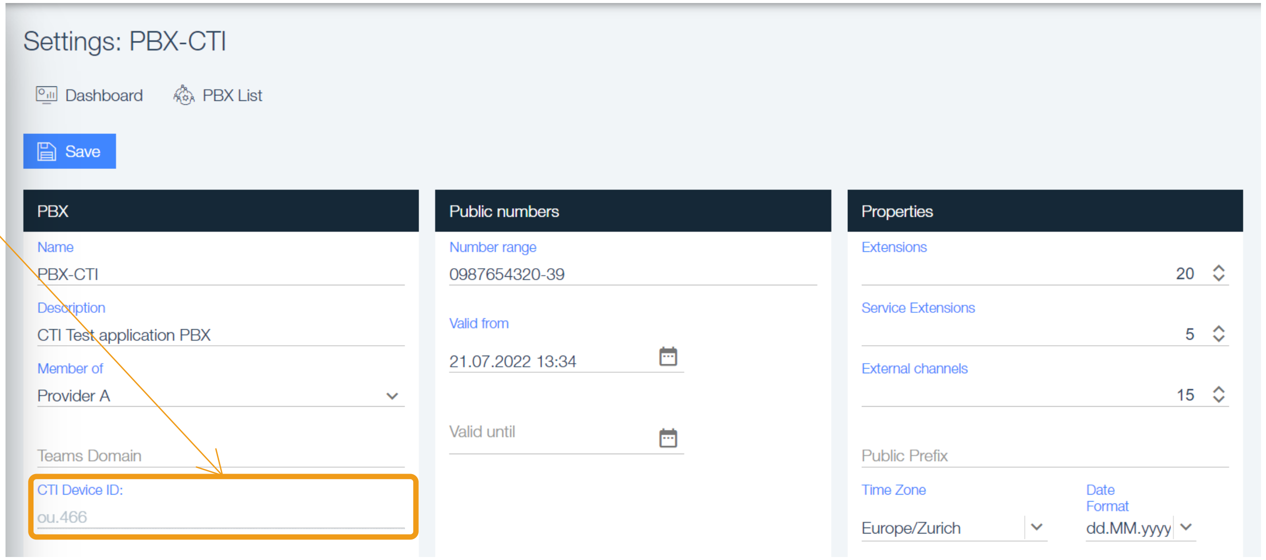
Monitored & Managed Phones and PBX Extensions

PBX Extension: 21 "Agent 1"
CTI Device ID `term.259`

PBX Extension: 22 "Agent 2"
CTI Device ID `term.260`

Service Extension: 40 "Ringing IVR"
CTI Device ID `ou.466`

- Obtain the PBX CTI Device ID for creating the CTI Device Reference of the managed service extensions:
  > Menu: PBX
    > Sub-Menu: PBX Settings



- Create the CTI device ID for a service extension:

Service Extension: 40 "Ringing IVR"
CTI Device ID `ou.466`

CTI Device References `ctiDeviceId: sip:40@ou.466`

5. CONFIGURE THE CTI DEVICE REFERENCES

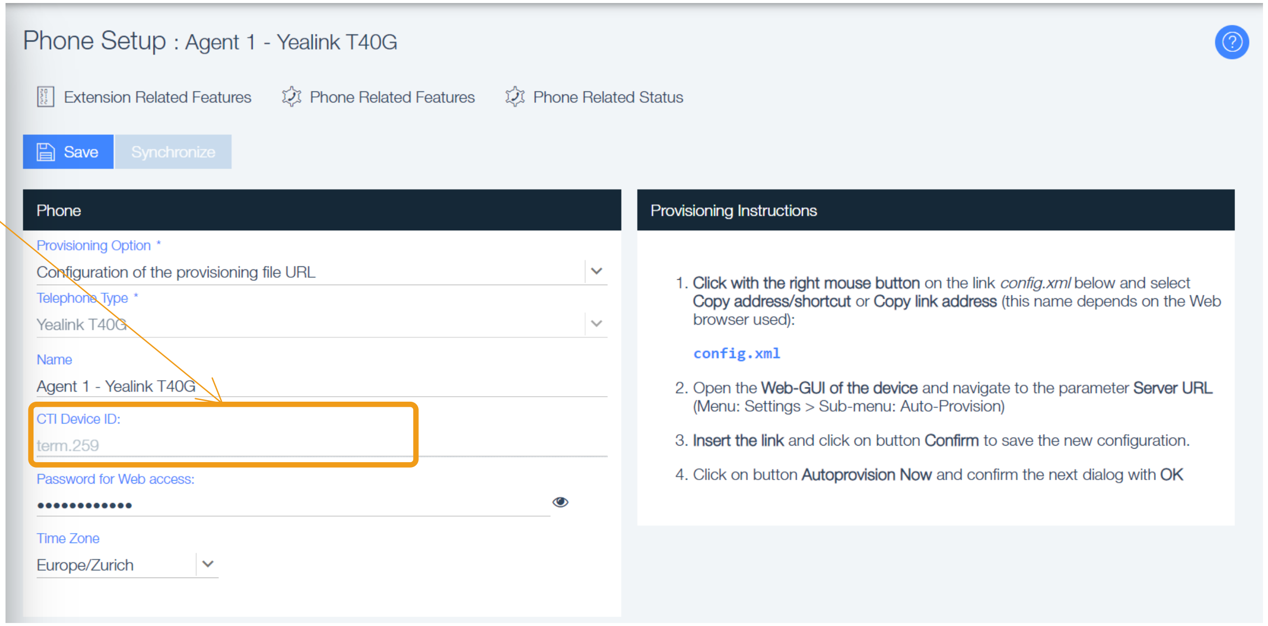
B. Obtain the CTI Device ID's of the managed phones.

Customer PBX

Monitored & Managed Phones and PBX Extensions

PBX Extension: 21 "Agent 1"	
CTI Device ID	`term.259`
PBX Extension: 22 "Agent 2"	
CTI Device ID	`term.260`
Service Extension: 40 "Ringing IVR"	
CTI Device ID	`ou.466`

- Obtain the PBX CTI Device ID for creating the CTI Device Reference of the managed phones:
  - > Menu: PBX
    - > Sub-Menu: Phones
      - > Select the row with the desired phone name
        - > Click link: Phone Setup



- Create the CTI device ID for a phone:

PBX Extension: 21 "Agent 1"	
CTI Device ID	`term.259`

↓

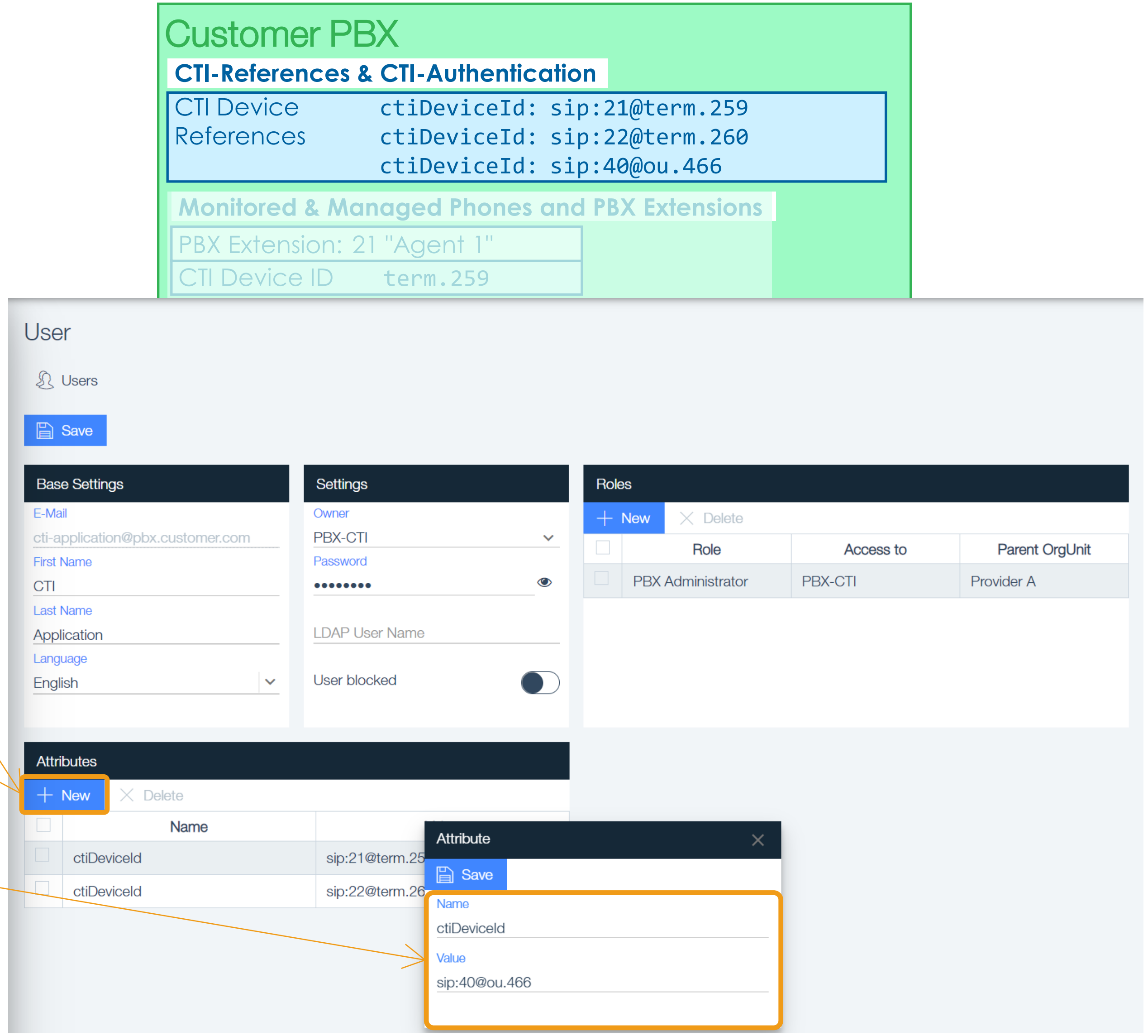
CTI Device References	`ctiDeviceId: sip:21@term.259`

5. CONFIGURE THE CTI DEVICE REFERENCES

C. Configure the prepared CTI device references in the CTI user account as attributes.

CTI Device References	ctiDeviceId: sip:21@term.259
	ctiDeviceId: sip:22@term.260
	ctiDeviceId: sip:40@ou.466

- Configure for each CTI device an own attribute:
  - > Menu: Operator
    - > Sub-Menu: Users
      - > Select the CTI user account
        - > At tile "Attributes" click button: + New
          - > Configure in the dialog "Attribute", for example:
            - Name: ctiDeviceId
            - Value: `CTI_DEVICE_ID`



SYNTAX CTI DEVICE REFERENCE

- The CTI device reference uniquely identifies an agent phone or service extension within the anSwitch V7

```
sip:NUMBER @ [ ou | term ].CTI_DEVICE_ID
```

NUMBER	[ou | term]	CTI_DEVICE_ID
The internal number of the phone or service extension.	The type of object: ou : service extension term : phone	Unique CTI-Device-ID of the phone or service extension instance.

Examples:

`sip:40@ou.466` References the PBX Service Extension "IVR Ringing" with the internal number 40 and the System-ID 452 of the PBX.

`sip:21@term.259` References the Agent 1 phone with internal number 21 and System-ID 6224 of the phone.
Note:
Agent 1 may have several phones. If needed reference each phone individually.

6. CONFIGURE BARGE-IN/WHISPER/LISTEN FOR A SUPERVISOR

- A supervisor may join into a running connection by calling the agent:
  - Barge In
    Join into the conversation. The supervisor can talk to both parties to both parties of the existing call.
  - Whisper
    Only the destination of the supervisor-call can hear the supervisor. The other party doesn't hear the supervisor's whispering.
  - Listen
    Just listen to the conversation. No party of the existing call can her the supervisor.

  - To use these features the caller (supervisor) needs special permission attributes in his OrgUnit PBX Extension.

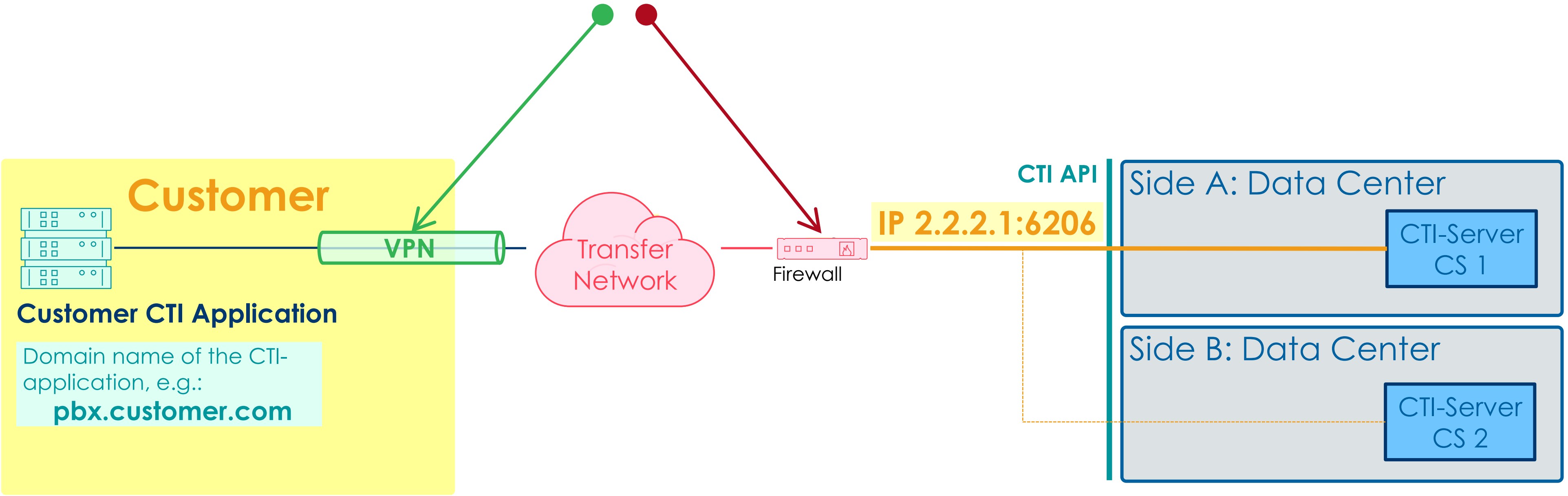
Attribute Name	Value	Example	Description
Supervisor Barge-In/Whisper/Listen			
supervisorBargeIn	[BEEP | SILENT]		The supervisor can talk to both parties of the call.
supervisorWhisper	[BEEP | SILENT]		Only the destination the supervisor is calling in can hear the supervisor. The other party doesn't hear the supervisor's whispering.
supervisorListen	[BEEP | SILENT]		Just listen to the conversation. No party of the existing call can her the supervisor.

6. CONFIGURE CALL RECORDING

- The conversation of an agent can be recorded.
  - The feature needs configuration concerning call recording, beep tone, audio file transfer script to use.
    Note:
      Recorded audio files are not stored on the anSwitch V7! They must be transferred to an external library.
  - For the programming and installation of the audio file transfer script the support of Aarenet system engineers is needed.
  - To use this feature, it must be enabled on the desired OrgUnit level, e.g.:
    - In the PBX Extension OrgUnit of the agent.
    - On the PBX OrgUnit level to enable it for all PBX Extensions,
  - For details about the feature call recording, see the training documentation: "Call Recording" (training_as7_709_call_recording)

7. CHECK AND ADJUST THE IP NETWORK & SECURITY ASPECTS

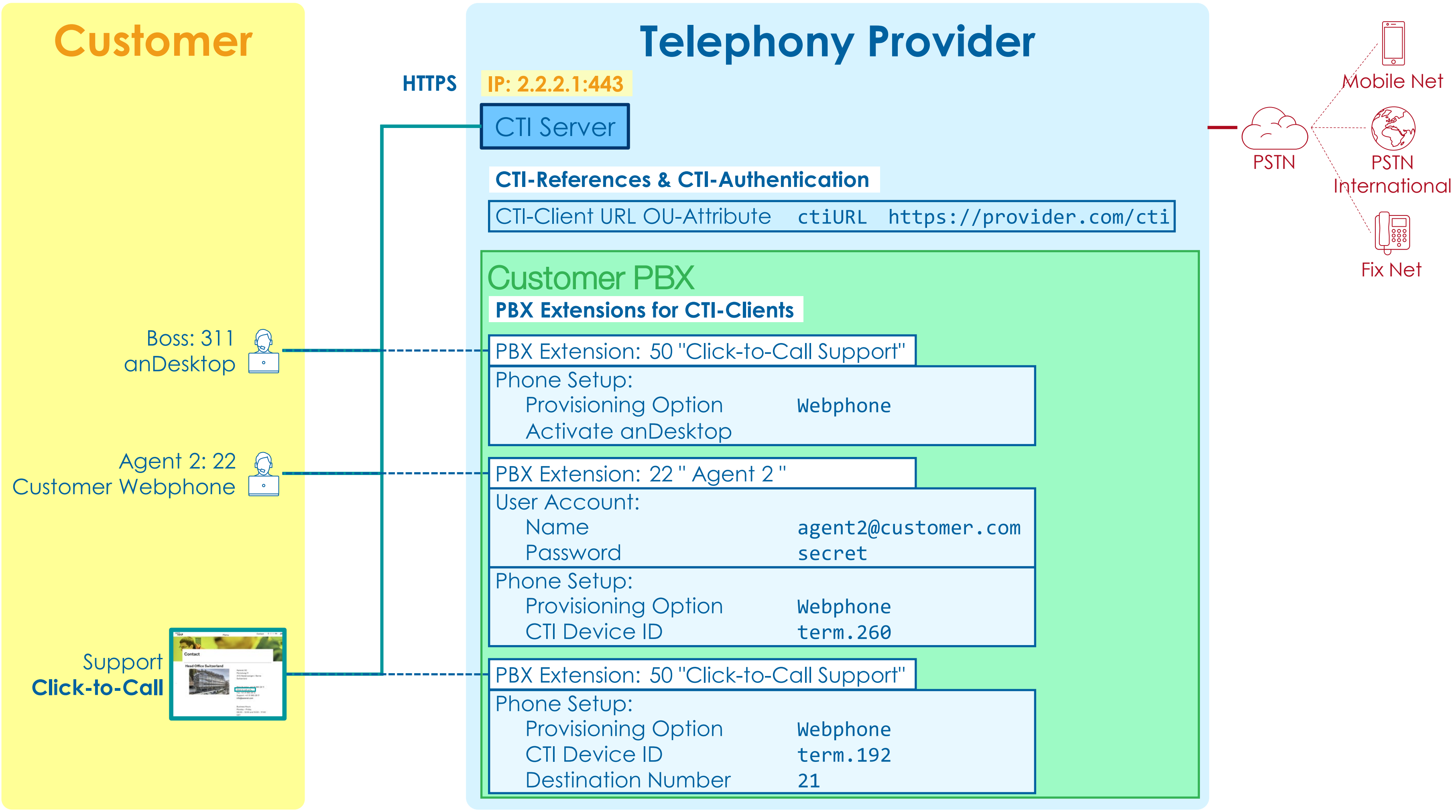
- Check and adjust the IP network & security aspects
  - Make sure that the IP network is ready.
  - For inputs see section "CTI API Related IT & Security Considerations"



3 PBX CONFIGURATION FOR INTERACTING WITH A CTI-CLIENT

OVERVIEW INTERWORKING CTI-APPLICATION WITH THE PBX

- This type of interworking with the PBX uses the anCTI JavaScript library.



1. DEFINE THE CTI-CLIENT URL

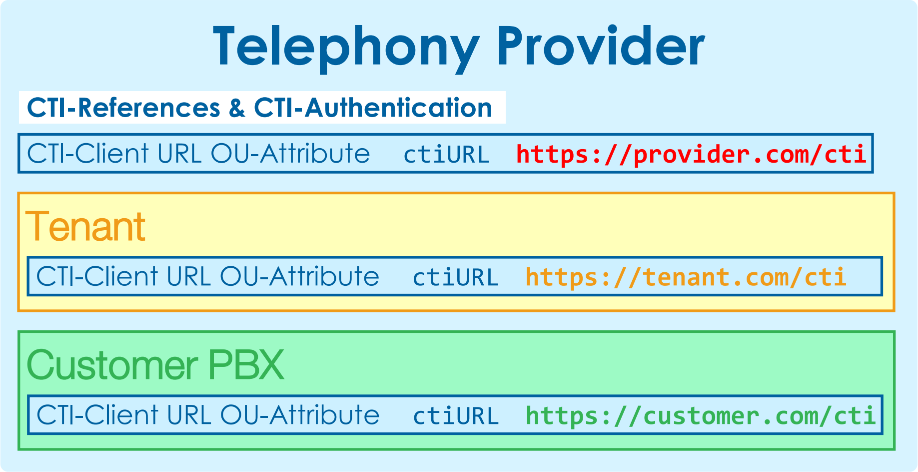
- Define the CTI-client URL.
  - The URL of the CTI-client URL can be different for the individual OrgUnit levels, e.g.:
    - System
    - Tenant
    - PBX
  - At provisioning time of the anSwitch V7 a CTI-client URL will be configured by default on system level.
    - ➔ For most situations, this system default CTI-client URL is sufficient.
    - ➔ Check with your telephony provider for the URL.

- Define optional tenant/customer specific URLs:
  - ☐ Define the URL and OrgUnit.
  - ☐ Provide certificates if needed.

2. CONFIGURE THE CTI-CLIENT URL

- Configure the CTI-Client URL at the requested OrgUnit level.
  - System
  - Tenant
  - PBX

Name	Value	Remark / Example
Attribute: ctiUrl	https://......./cti	Example URL: https://provider.com/cti

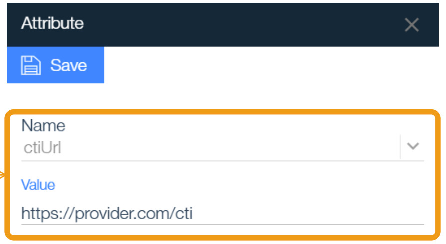


a. If customer certificates must be installed, then contact the Aarenet Support.

b. Configure the CTI-client URL for the desired OrgUnit:

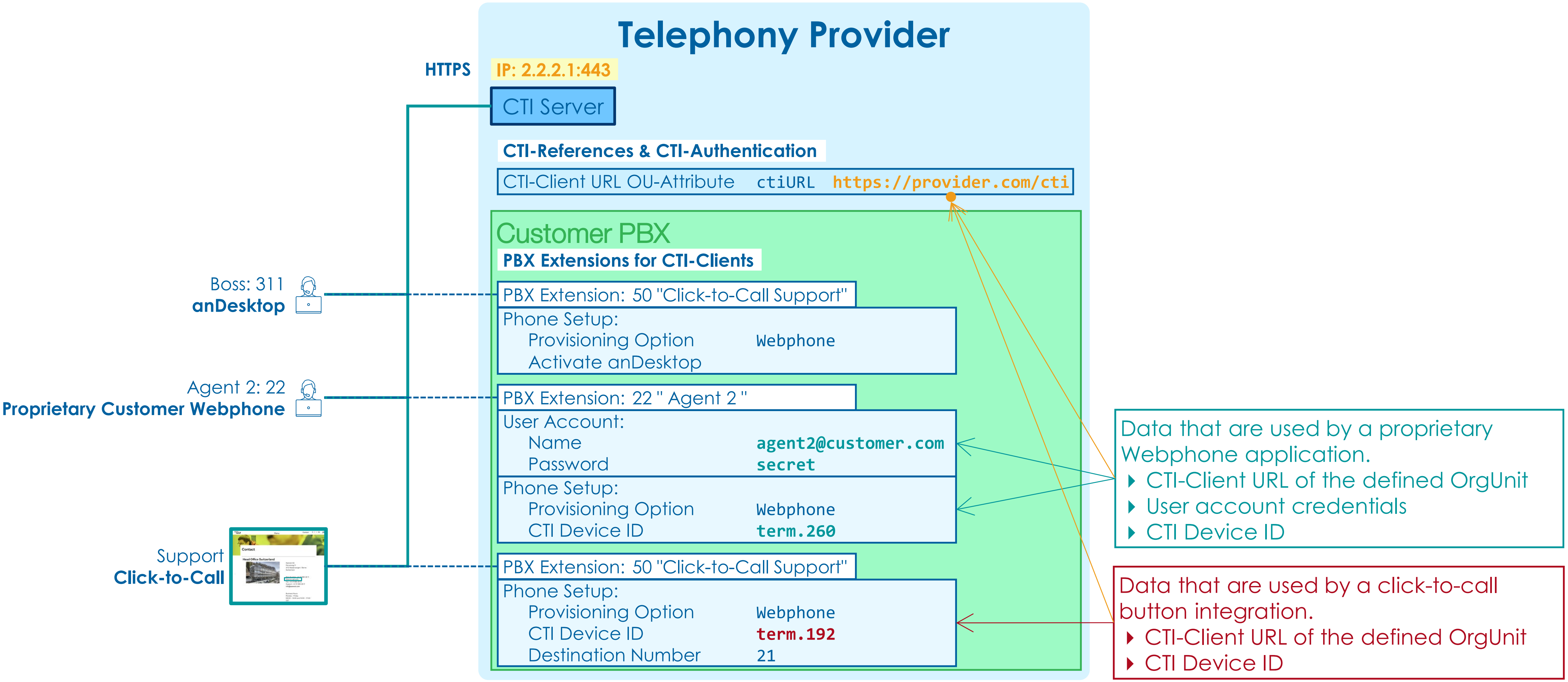
> Click menu: Operations
> Click sub-menu: Organization Units
> Select the requested OrgUnit

> Add and configure new attribute:
- Name: ctiUrl
- Value: CTI_URL_VALUE



OVERVIEW OF NEEDED PBX PARAMETER VALUES FOR CTI-CLIENTS

- Overview of needed PBX parameter values for different CTI-clients.



ASSIGN A CTI-CLIENT TO A PBX EXTENSION

Assigning anDesktop

See training documentation "anDesktop PC Client" (training_as7_805_andesktop_e06)

Assigning Click-to-Call

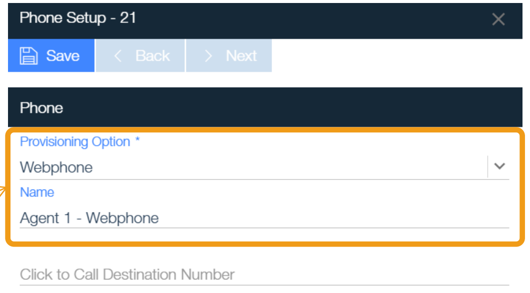
See training documentation "Click-To-Call" (training_as7_806_click_to_call_e06)

Assigning a proprietary Webphone

a. Assign the Web phone client as "Webphone" to the PBX Extension

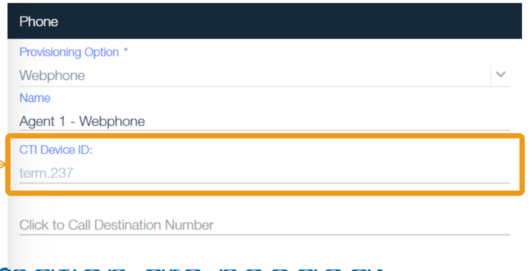
> Click menu: PBX
  > Click sub-menu: Extensions
    > Select the desired PBX extension

> Tile 'Assigned Phones' click button: + New Phone
  > Configure:
  - Select Provisioning Option: Webphone
  - Name: Define the name for the terminal



b. Check the assigned unique CTI device ID:

> Find the page of the newly created Webphone: Phone Setup
  > Check the parameter: CTI Device ID



c. For the configuration of the CTI-client the following information are needed:
- CTI-Client URL of the defined OrgUnit
- The username and password of the user assigned to the PBX Extension.
- The CTI Device ID

4 USING THE CTI API WITH PURE CSTA

NOTE ON USING THE CTI API WITH PURE CSTA

- Here you can find detailed instructions and CTI-client examples for using the CTI API with Pure CSTA.
  - Directly on your system:
    https://provider.com/cti/ancti/docs/ancti.html
    or here (not always online):
    https://as7poc.aarenet.com/cti/ancti/docs/ancti.html

Note	- SW engineering skills are required for using the pure CSTA CTI API either to customize an existing CTI-application or to develop a new CTI-application.
	- Software development knowledge is required in particular:
	  - For structured thinking
	  - Understand and develop status machines
	  - Modelling of message flows and message contents
	- A longer process for familiarization, understanding and application with ECMA CSTA standard must be expected.

4.1 OVERVIEW CSTA STANDARD & SUPPORTED CSTA SERVICES

OVERVIEW CSTA STANDARD

- The Call Control anCTI API is based on the standardized ECMA CSTA protocol defined in ECMA-269 / ECMA-323:
  https://www.ecma-international.org/publications-and-standards/standards/ecma-269
  https://www.ecma-international.org/publications-and-standards/standards/ecma-323

Collection of CSTA standard documents:

Document Title	ECMA Publication	ISO/IEC Publication	ETSI Publication
Services for CSTA Phase III	**ECMA-269**	ISO/IEC 18051	ETSI TS 102 173
XML Protocol for CSTA Phase III	**ECMA-323**	ISO/IEC 18056	ETSI TS 102 174
Application Session Services (used in ECMA-269)	ECMA-354	ISO/IEC 22534	ETSI TS 102 344
Scenarios for Computer Supported Telecommunications Applications (CSTA) Phase III	ECMA TR/82		
Using ECMA-323 (CSTA XML) in a Voice Browser Environment	ECMA TR/85	ISO/IEC TR 18057	ETSI TR 102 171

OVERVIEW CSTA STANDARD

- ECMA based information:
  - These Technical Report provides example call scenarios based upon Phase III of Services for Computer Supported Telecommunications Applications (CSTA). All of the Standards and Technical Reports in the Suite are based on practical experience of ECMA member companies and each one represents a pragmatic and widely-based consensus.
  - ECMA Technical Report: "Scenarios for Computer Supported Telecommunications Applications (CSTA) Phase III", ECMA TR/82
  - ECMA Technical Report: "Using ECMA-323 (CSTA XML) in a Voice Browser Environment", ECMA TR/85

SUPPORTED CSTA SERVICES & PARAMETERS BY THE ANSWITCH V7

CSTA Services supported:

Device Service

- MakeCall
- JoinCall
- MonitorStart
- MonitorStop
- DirectedPickupCall
- SetForwarding
- GetForwarding
- SetDoNotDisturb
- GetDoNotDisturb
- SnapshotDevice
- SnapshotCall
- GetPresenceState

Call Services

- AnswerCall
- ClearConnection
- SingleStepTransferCall
- HoldCall
- RetrieveCall
- GenerateDigits
- DeflectCall
- DirectedPickupCall
- TransferCall
- ConferenceCall
- PlayMessage
- RecordMessage
- Stop

CSTA Events emitted:

Device Events

- PresenceStateEvent

Call Events

- ServiceInitiatedEvent
- FailedEvent
- ConnectionClearedEvent
- OriginatedEvent
- DeliveredEvent
- DivertedEvent
- EstablishedEvent
- HeldEvent
- RetrievedEvent
- TransferredEvent
- DtmfDetectedEvent
- StopEvent

CSTA Parameters supported:

- According ECMA-269 the **mandatory** parameters of a service or event are supported
- Optional parameters are not supported.

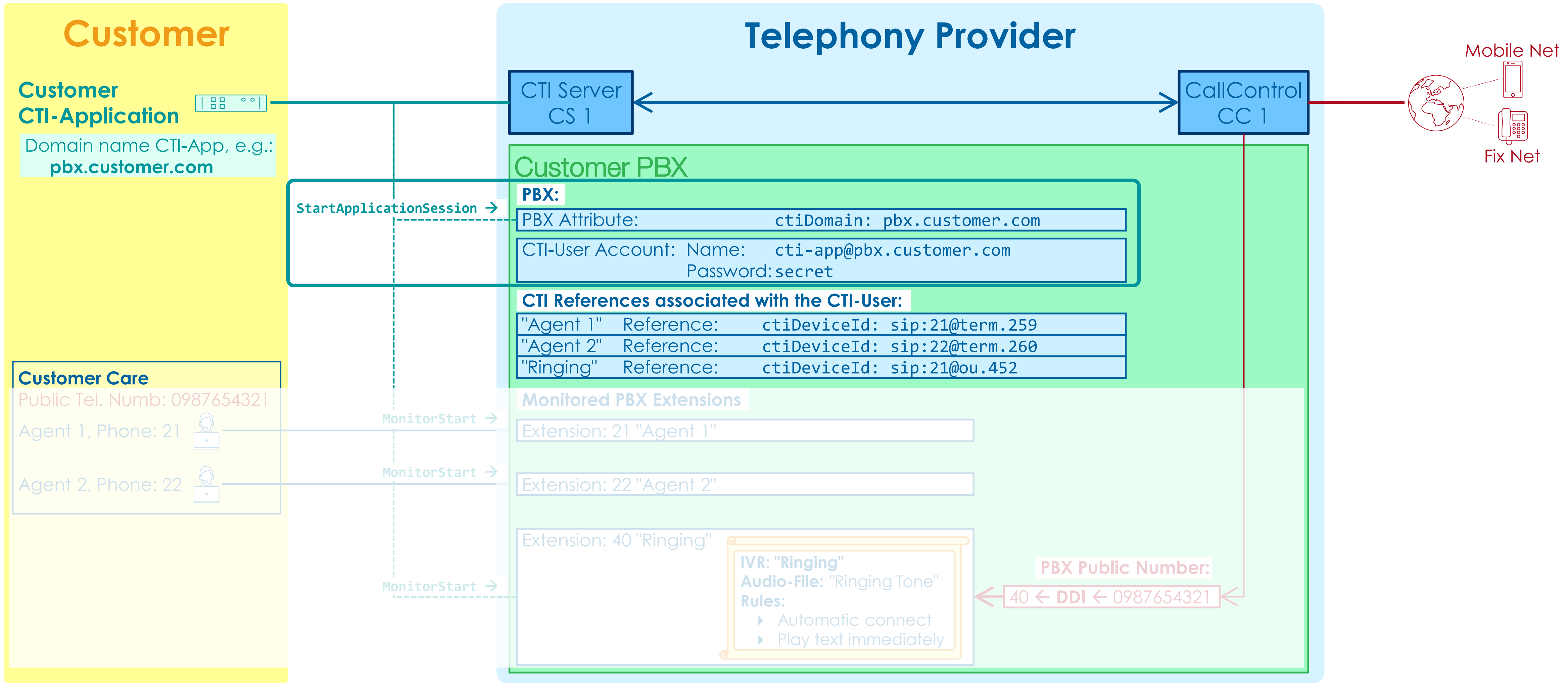
Source: ECMA-269

Table 15-4 Monitor Start—Service Request

Parameter Name	Type	M/ O/C	Description
monitorObject	Choice Structure	$\boxed{\text{M}}$	Specifies the monitor object of a call or device to be monitored. This shall be one of the following choices: • call (ConnectionID) - Specifies a call (connection) object. • device (DeviceID) - Specifies a device object. See Functional Requirement #5.
requestedMonitorFilter	MonitorFilter	O	This parameter specifies the requested set of events to be filtered out (not sent) by the switching function. It is a

4.2 EXAMPLE MESSAGE FLOW START THE APPLICATION SESSION

OVERVIEW EXAMPLE MESSAGE FLOW START THE APPLICATION SESSION



MESSAGE FLOW START THE APPLICATION SESSION

- Once the TCP connection is open the CTI application first needs to send a `StartApplicationSession` message to authenticate itself on the anSwitch CTI server, e.g.:

```
<?xml version="1.0" encoding="utf-8"?>
<StartApplicationSession xmlns="http://www.ecma-international.org/standards/ecma-
  <applicationInfo>
    <applicationID>appID</applicationID>
    <applicationSpecificInfo>
      <username>cti-app@pbx.customer.com</username>
      <password>secret</password>
    </applicationSpecificInfo>
  </applicationInfo>
  <requestedProtocolVersions>
    <protocolVersion>http://www.ecma-international.org/standards/ecma-323/csta/ed6</protocolVersion>
  </requestedProtocolVersions>
</StartApplicationSession>
```

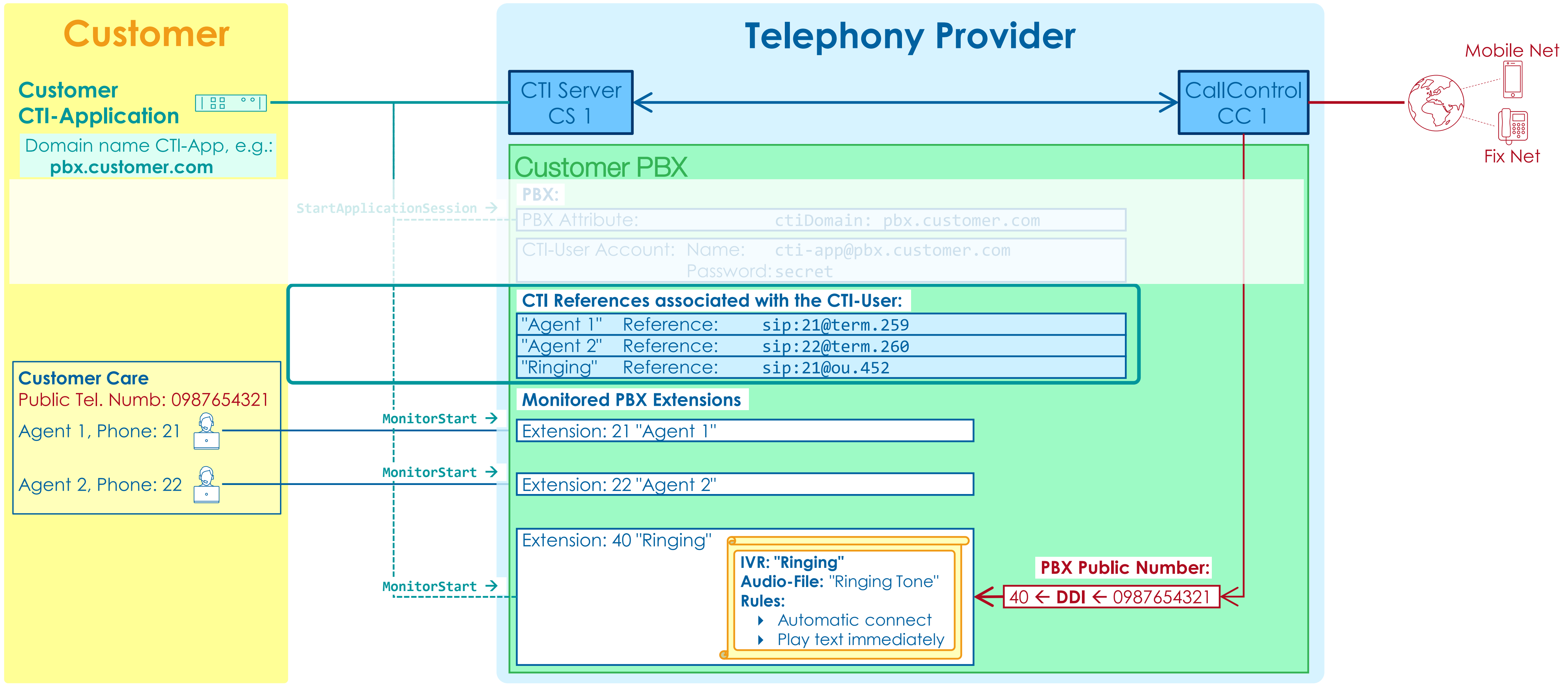
"applicationID":
- The value of the application ID is currently not used by the CTI-Server

- The CTI Server answers with a `StartApplicationSessionPosResponse` message:

```
<?xml version="1.0" encoding="utf-8"?>
<StartApplicationSessionPosResponse xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <sessionID>5</sessionID>
  <actualProtocolVersion>http://www.ecma-international.org/standards/ecma-323/csta/ed6</actualProtocolVersion>
  <actualSessionDuration>315360000</actualSessionDuration>
</StartApplicationSessionPosResponse>
```

4.3 EXAMPLE MESSAGE FLOW START DEVICE MONITORING

OVERVIEW EXAMPLE MESSAGE FLOW START DEVICE MONITORING



START MONITORING DEVICES

- Once the application is authenticated it can start monitoring devices.
- The CTI application sends for each SIP device it wants to monitor a **`MonitorStart`** message, e.g.:

```xml
<?xml version="1.0" encoding="utf-8"?>
<MonitorStart xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <monitorObject>
    <deviceObject>sip:21@term.259</deviceObject>
  </monitorObject>
</MonitorStart>
```

- The CTI Server checks if the CTI application is allowed to monitor this device and answers with a **`monitorStartResponse`** message:

```xml
<?xml version="1.0" encoding="utf-8"?>
<MonitorStartResponse xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <monitorCrossRefID>235</monitorCrossRefID>
</MonitorStartResponse>
```

EVENTS OF MONITORED DEVICES

- Once the CTI-application is monitoring a device it gets CSTA events from the CTI-Server, e.g.:


```xml
<?xml version="1.0" encoding="utf-8"?>
<DeliveredEvent xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <monitorCrossRefID>235</monitorCrossRefID>
  <connection>
    <callID>3</callID>
    <deviceID>sip:21@term.259</deviceID>
  </connection>
  <alertingDevice>
    <deviceIdentifier>sip:21@term.259</deviceIdentifier>
  </alertingDevice>
  <callingDevice>
    <deviceIdentifier>N<+41987654321>Aarenet AG</deviceIdentifier>
  </callingDevice>
  <calledDevice>
    <deviceIdentifier>sip:21@term.259</deviceIdentifier>
  </calledDevice>
  <lastRedirectionDevice>
    <notKnown/>
  </lastRedirectionDevice>
  <localConnectionInfo>alerting</localConnectionInfo>
  <cause>normal</cause>
</DeliveredEvent>
```

SUPPORTED CSTA MONITORING EVENTS

- Supported CSTA Monitoring Events
  - Call Events
    - ServiceInitiatedEvent
    - FailedEvent
    - ConnectionClearedEvent
    - OriginatedEvent
    - DeliveredEvent
    - DivertedEvent
    - EstablishedEvent
    - HeldEvent
    - RetrievedEvent
    - TransferredEvent
    - DtmfDetectedEvent
  - Device Events
    - PresenceStateEvent

Note	Details and examples about this services can be found in the ECMA doc "Using ECMA-323 (CSTA XML) in a Voice Browser Environment", ECMA TR/85.

SPECIAL NOTES ABOUT EVENTS OF MONITORED DEVICES

- When the CTI-application monitors the calling and the called device of a call, it gets events for both devices independently. But as it is the same call, the `callID` will be the same on both sides.
- The `DtmfDetectedEvent` is also treated specially as it is emitted on both devices associated to the call. This allows an IVR-application of the customer to react on DTMF keypresses that were emitted by the calling device.

4.4 EXAMPLE START OUTGOING CALL

EXAMPLE START OUTGOING CALL

- Start an outgoing video call for caller 21 to destination 21 with **`MakeCall`**:

```
<MakeCall xmlns='http://www.ecma-international.org/standards/ecma-323/csta/ed6'>
    <callingDevice>sip:21@term.259</callingDevice>
    <calledDirectoryNumber>22</calledDirectoryNumber>
    <autoOriginate>doNotPrompt</autoOriginate>
    <mediaCallCharacteristics>
        <mediaClass>
            <voice>true</voice>
            <image>true</image>
        </mediaClass>
    </mediaCallCharacteristics>
</MakeCall>
```

- By default, all calls are normal voice calls. The **`MakeCall`** service also allows to specify the media-characteristics using the **`mediaCallCharacteristics`** property.
  Currently following media-classes are supported:
  - voice
  - image

Note	Calls without voice can only be answered by phones with video capabilities! Standard phones will reject the call if there is no audio-stream in the initial INVITE. Therefore it is suggested to enable voice if the **`mediaCallCharacteristics`** are specified.

EXAMPLE REDEFINE THE CALLER ID FOR AN OUTGOING CALL

- The CSTA specification doesn't provide a standard property for defining a different caller id.
- The anSwitch V7 provides a proprietary property for defining a different caller id:
  - The element `privateData` is used with the proprietary `callingNumber` element.
- The following example shows how to define a different caller id: `080099887766`

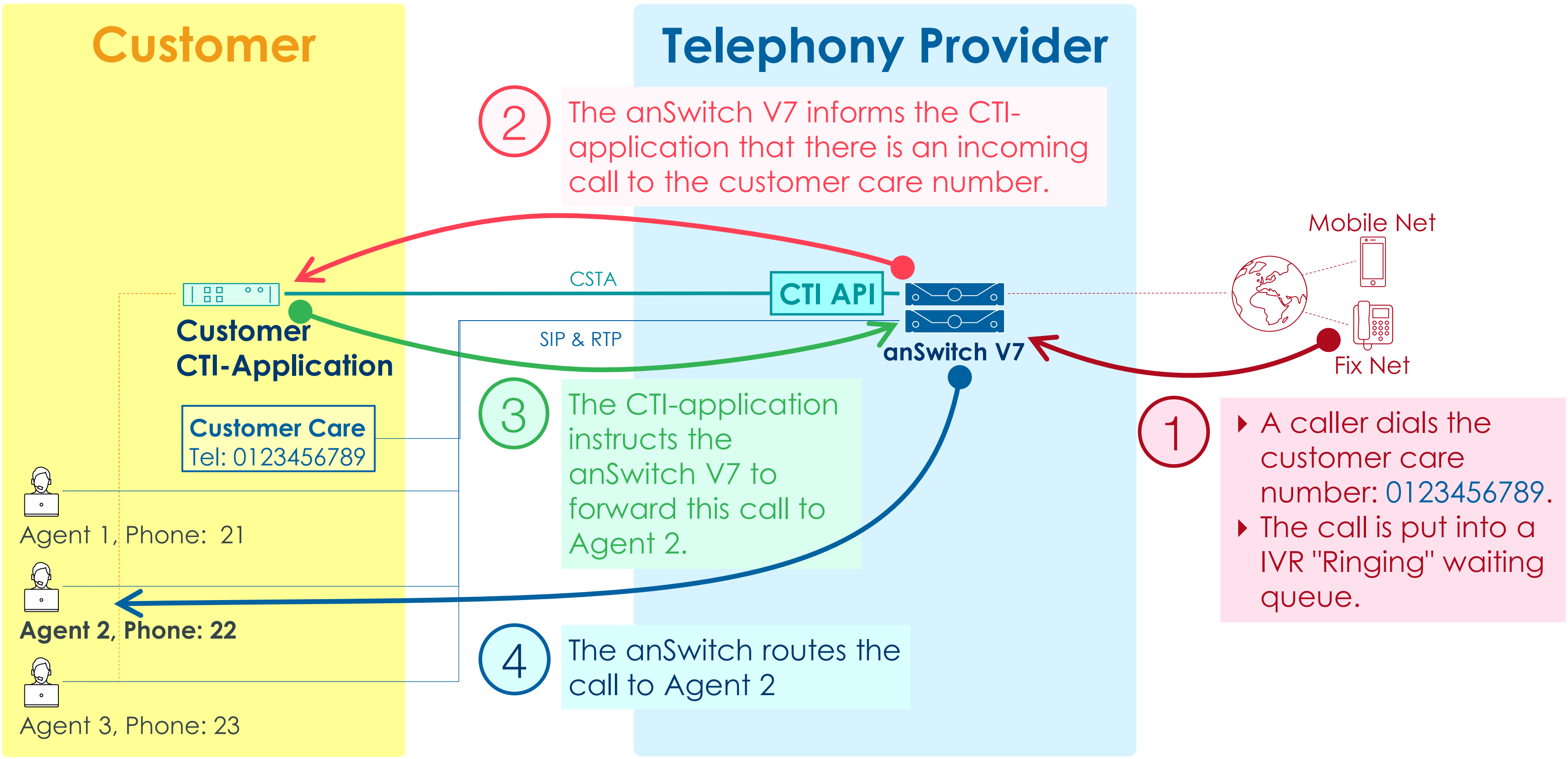
```
<MakeCall>
  <callingDevice>sip:21@term.259</callingDevice>
  <calledDirectoryNumber>0776665544</calledDirectoryNumber>
  <privateData>
    <callingNumber>080099887766</callingNumber>
  </privateData>
</MakeCall>
```

- For using this feature, it is mandatory to define the OrgUnit attribute "clipSignaling" in the desired OrgUnit e.g.: PBX, department that contains all agents, etc..

OrgUnit Attribute Name	Value	Remark
clipSignaling	[ANY | PBX]	ANY: Every number can be specified and will be used as preferred public number (like "Special Arrangement" in anSwitch V6) PBX: Every public number of the PBX can be used.

4.5 EXAMPLE INCOMING CALL FORWARDED TO AN AGENT

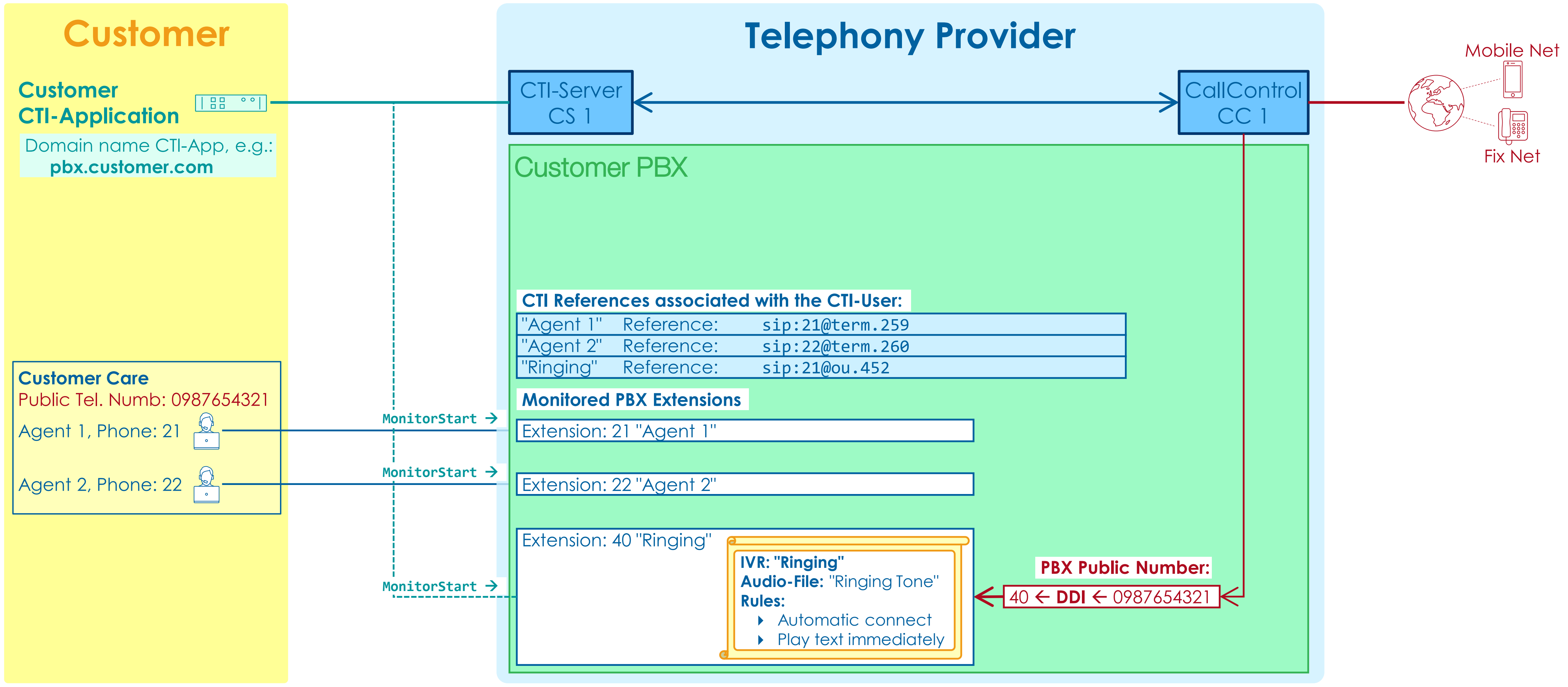
OVERVIEW OF THE SCENARIO OF AN INCOMING CALL



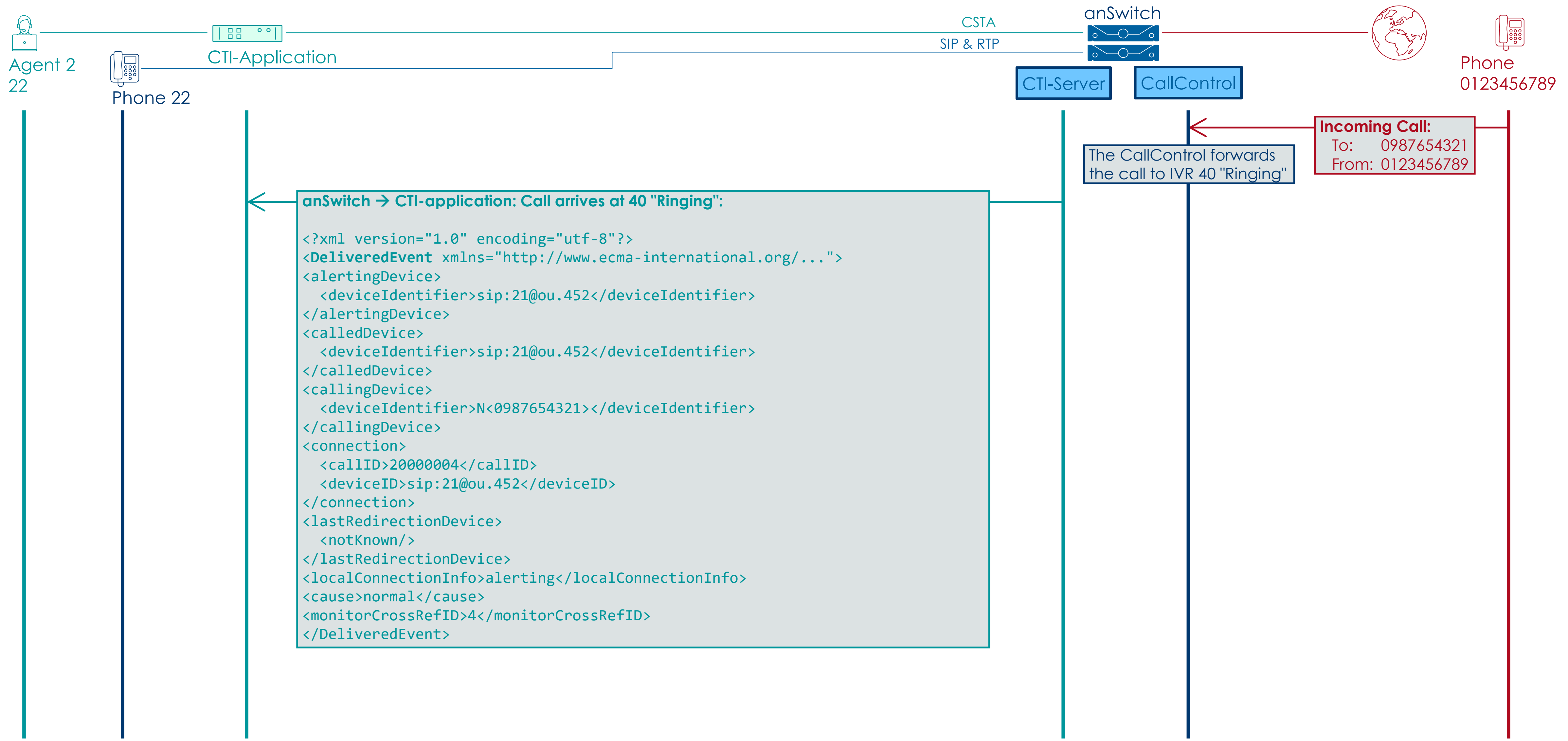
SCENARIO INCOMING CALL FORWARDED TO AGENT 2

- Scenario in wording:
  1. Incoming call from the PSTN is routed by the CallControl to the IVR "Ringing"
  2. The calling party is connected to the IVR "Ringing" and hears ringing tone.
  3. The CTI-Server sends a **`DeliveredEvent`** event to the CTI-application that number 40 is called.
  4. The CTI-Server sends a **`EstablishedEvent`** event to the CTI-application that 40 made connection to the IVR "Ringing".
  5. The CTI-application evaluates where to transfer the call.
  6. The CTI-application orders with the **`SingleStepTransferCall`** service to forward the call to Agent 2 with the number 22.
  7. The CTI-Server confirms with a **`TransferedEvent`** that it transferred the call to number 22

OVERVIEW OF THE PBX SETUP AND REFERENCES



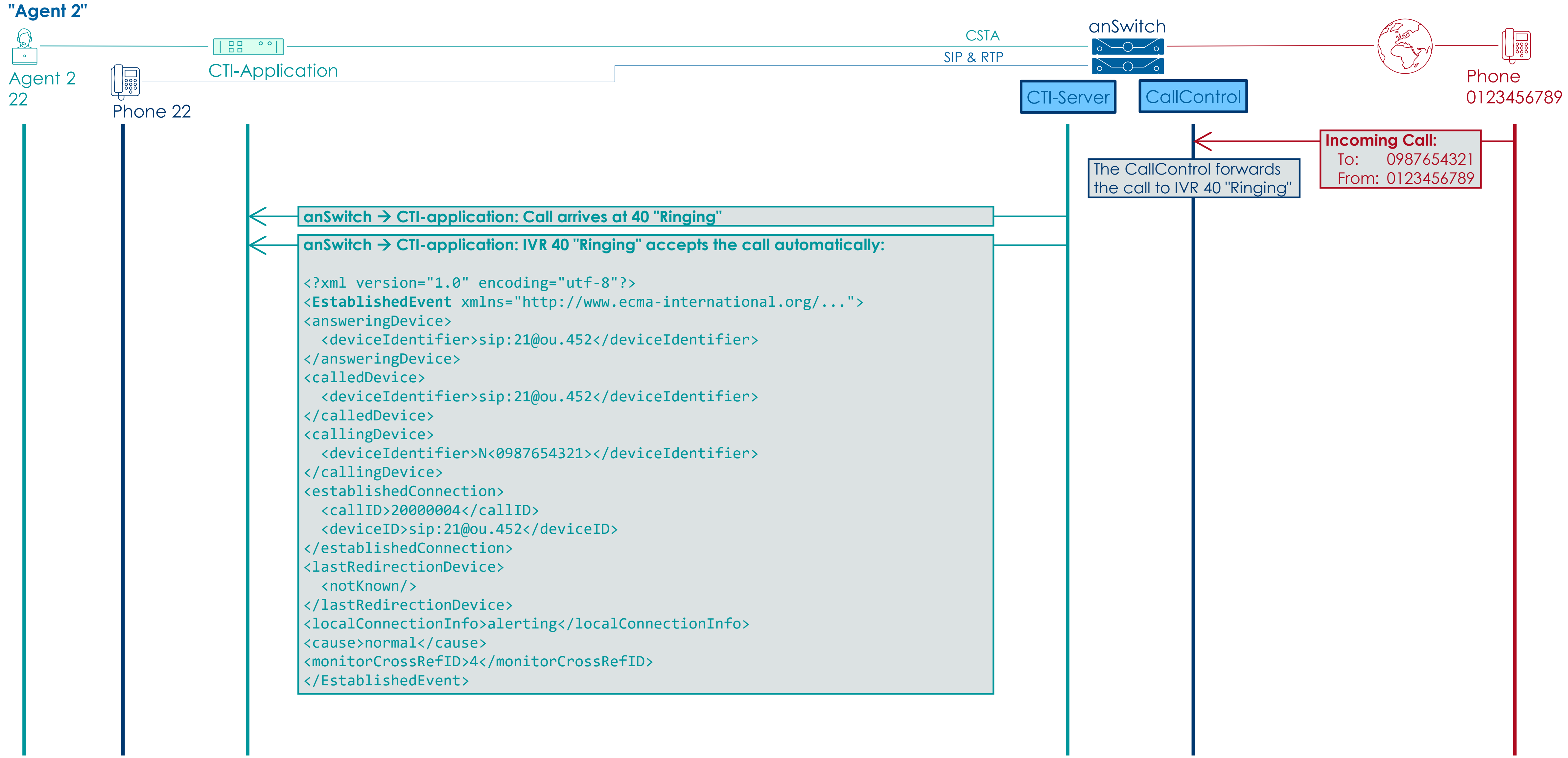
INCOMING CALL SIGNALED TO THE CTI-APPLICATION



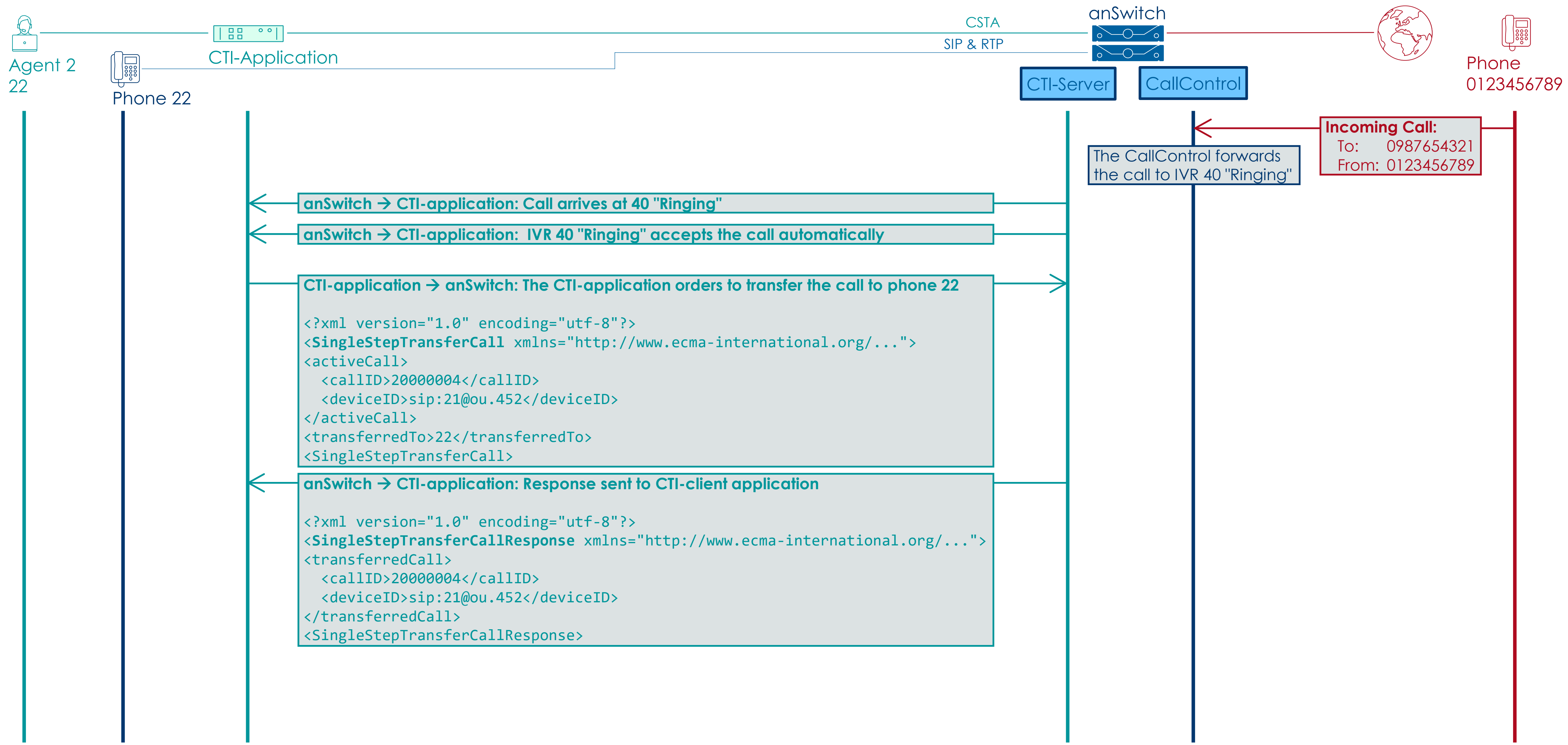
anSwitch → CTI-application: Call arrives at 40 "Ringing":


```xml
<?xml version="1.0" encoding="utf-8"?>
<DeliveredEvent xmlns="http://www.ecma-international.org/...">
<alertingDevice>
  <deviceIdentifier>sip:21@ou.452</deviceIdentifier>
</alertingDevice>
<calledDevice>
  <deviceIdentifier>sip:21@ou.452</deviceIdentifier>
</calledDevice>
<callingDevice>
  <deviceIdentifier>N<0987654321></deviceIdentifier>
</callingDevice>
<connection>
  <callID>20000004</callID>
  <deviceID>sip:21@ou.452</deviceID>
</connection>
<lastRedirectionDevice>
  <notKnown/>
</lastRedirectionDevice>
<localConnectionInfo>alerting</localConnectionInfo>
<cause>normal</cause>
<monitorCrossRefID>4</monitorCrossRefID>
</DeliveredEvent>
```

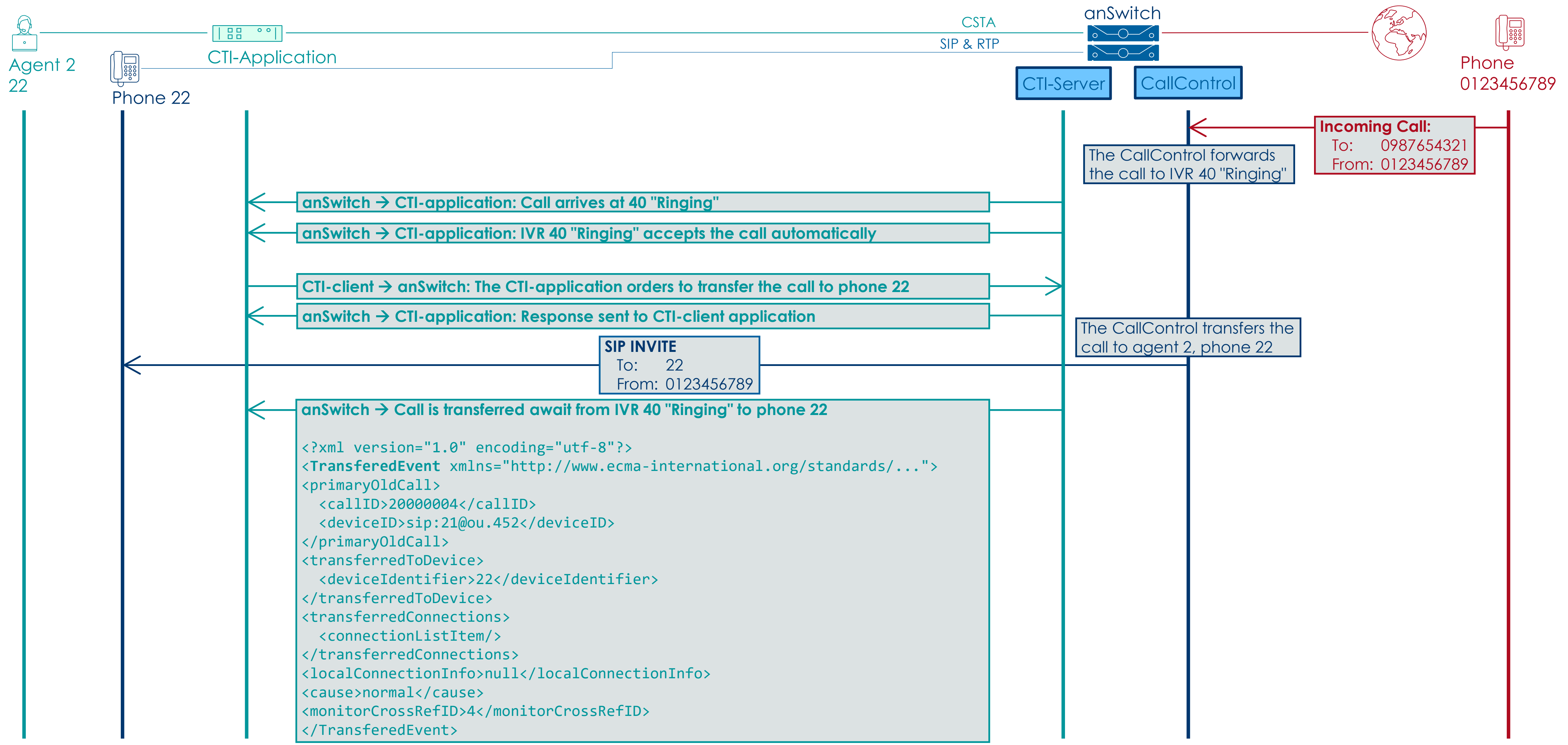
INCOMING CALL IS QUEUED IN IVR "RINGING"



THE CTI-APPLICATION ORDERS TO FORWARD THE CALL TO 22



THE CALLCONTROL TRANSFERS THE CALL TO 22



4.6 EXAMPLE HANDLING AUDIO FILES, WAITING MUSIC DURING ON-HOLD & ACD/IVR

BASIC CONCEPT OF MUSIC ON-HOLD & WAITING MUSIC

- ## Music On-Hold during call transfer:

  If a standard phone transfers a call, it normally puts the initial call on hold. This means the caller hears music-on-hold while waiting to be reconnected. The CTI API also uses the same concept on PBX Extensions and places the caller on hold.

- ## Music during Waiting in ACD/IVR:

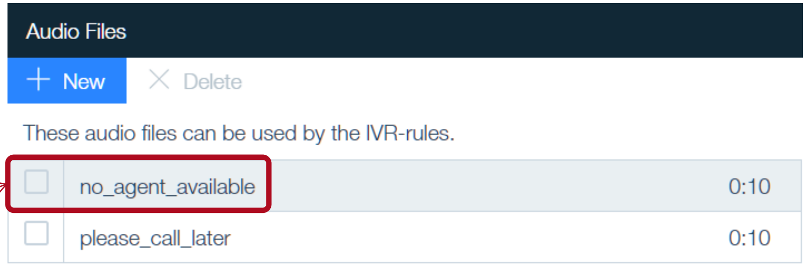
  The ACD and IVR feature extend this functionality as they explicitly allow to configure a waiting-music (if no waiting-music is configured the ring back tone is played). Even if the call is forwarded to several destinations in sequence, the caller still hears the waiting-music. The music on-hold is not played in this situation. The CTI server implementation reuses this concept if a call is transferred away from an ACD/IVR. The caller does not get music-on-hold while being transferred, but still listens to the waiting-music.

CONTROLLING TO PLAY AUDIO-FILES SERVICES

- The following CSTA services can be used for playing audio-files :
  - Start playing an audio file (`PlayMessage`)
  - Stop playing an audio file (`Stop`)

- Referencing the audio-file:

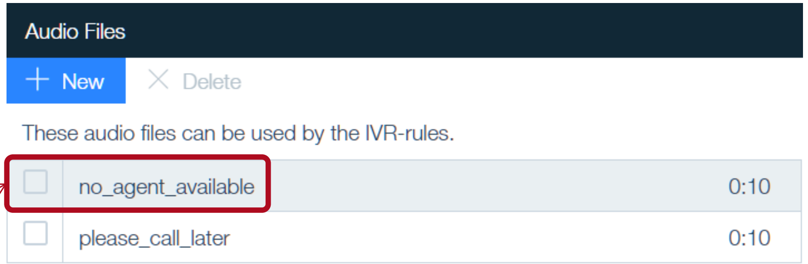
  IVRs and ACDs allow to upload audio-files in the Portal UI. The CTI application can use the `PlayMessage` service to play them.
  The files are referenced by their name given in the Portal UI.



EXAMPLE OF PLAY AUDIO-FILES

Referencing the audio-file:

IVRs and ACDs allow to upload audio-files in the Portal UI. The CTI application can use the `PlayMessage` service to play them.
The files are referenced by their name given in the Portal UI.



Start to play an audio-file with `PlayMessage`:

```xml
<?xml version="1.0" encoding="utf-8"?>
<PlayMessage xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <messageToBePlayed>no_agent_available</messageToBePlayed>
  <overConnection>
    <callID>3</callID>
    <deviceID>sip:21@term.259</deviceID>
  </overConnection>
</PlayMessage>
```

Stop playing an audio-file with `Stop`:

```xml
<?xml version="1.0" encoding="utf-8"?>
<Stop xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <messageToBeStopped>no_agent_available</messageToBeStopped>
  <connection>
    <callID>3</callID>
    <deviceID>sip:21@term.259</deviceID>
  </connection>
</Stop>
```

4.7 EXAMPLE HANDLING DO NOT DISTURB DND, CHECK CALL FORWARDING OF A DEVICE

CONTROLLING OF DEVICE SERVICES

- Beside the call-related services there are also device-related services e.g.:
  - Call Forwarding Configuration of a Device (`GetForwarding`, `SetForwarding`):
    Get and set the call forwarding configuration of a device.
  - Do Not Disturb DnD Configuration of a Device(`GetDoNotDisturb`, (`SetDoNotDisturb`):
    Get and set the do not disturb configuration of a device.

Note	Details and examples about this services can be found in the ECMA doc "XML Protocol for CSTA Phase III", ECMA-323

RETRIEVE THE CALL FORWARDING SETTINGS OF A DEVICE

- Retrieve the call forwarding settings of number 25:

```xml
<?xml version="1.0" encoding="utf-8"?>
<GetForwarding xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <device>sip:25@pbx.customer.com</device>
</GetForwarding>
```

- Possible response from the CTI-Server:

```xml
<?xml version="1.0" encoding="utf-8"?>
<GetForwardingResponse xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6">
  <forwardingList>
<forwardListItem>
      <forwardingType>forwardBusy</forwardingType>
      <forwardStatus>true</forwardStatus>
      <forwardDN>28</forwardDN>
    </forwardListItem>
    <forwardListItem>
      <forwardingType>forwardNoAns</forwardingType>
      <forwardStatus>true</forwardStatus>
      <forwardDN>*86</forwardDN>
      <ringDuration>15</ringDuration>
    </forwardListItem>
  </forwardingList>
</GetForwardingResponse>
```

ACTIVATE DO NOT DISTURB DND

- Activate Do Not Disturb DND for number 25

```
<?xml version="1.0" encoding="UTF-8"?>
<SetDoNotDisturb xmlns=”http://www.ecma-international.org/standards/ecma-323/csta/ed6”>
  <device>sip:25@pbx.customer.com</device>
  <doNotDisturbOn>false</doNotDisturbOn>
</SetDoNotDisturb>
```

- Activation response from the CTI-Server:

```
<?xml version="1.0" encoding="UTF-8"?>
<SetDoNotDisturbResponse xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed6"/>
```

4.8 SIGNALING "LAST USED" TERMINAL

SIGNALING "LAST USED" TERMINAL

- If an agent's PBX Extension has several terminals registered the CTI-application is not aware of this.
- The CTI-application routes an incoming call to the number of an PBX Extension (agent). There is no information for the CallControl to which phone the call shall be signaled.
- In this situation the CallControl will signal the incoming call to the "last used" terminal of this PBX Extension.
- If the agent uses for an outgoing call another phone associated with his PBX Extension, CallControl uses this terminal as "last used" for the next incoming call.

4.9 GET THE CDR OF A CALL VIA CSTA & REST API

GET THE CDR OF A CALL VIA CSTA & REST API

- For retrieving the CDR of a connection via a CTI application a unique ID is delivered by the CTI Server with the CSTA "EstablishedEvent". The needed call ID is delivered in CSTA Parameter "globallyUniqueCallLinkageID":

```
<EstablishedEvent xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed3">
  ...
 <callLinkageData>
  <globalCallData>
   <globalCallLinkageID>
    <globallyUniqueCallLinkageID>cc1xy-2787a67fe921f02d</globallyUniqueCallLinkageID>
   </globalCallLinkageID>
  </globalCallData>
 </callLinkageData>
</EstablishedEvent>
```

- If using the anCTI JavaScript library check the property "globallyUniqueCallLinkageID".

- The delivered ID can then be used with REST API CDR query.

```
GET https://DOMAIN_REST/rest/cdrs?where=callId.eq("cc1xy-2787a67fe921f02d")
```

5 USING THE CTI API WITH THE ANCTI JAVASCRIPT LIBRARY

NOTE ON USING THE CTI API WITH ANCTI JAVASCRIPT LIBRARY

- Here you can find detailed instructions and CTI-client examples for using the anCTI JavaScript library.

  - Access anCTI JavaScript library documentation:
    https://PROVIDER_DOMAIN/cti/ancti/docs/ancti.html
    or here (but not always online):
    https://as7poc.aarenet.com/cti/ancti/docs/ancti.html

  - Access anCTI JavaScript examples:
    https://PROVIDER_DOMAIN/cti/examples/README.html
    or here (but not always online):
    https://as7poc.aarenet.com/cti/examples/README.html

5.1 OVERVIEW ANCTI LIBRARY SUPPORTED METHODS & EVENTS

SUPPORTED ANCTI API METHODS & EVENTS

Supported Methods:

Agent Services

- StartApplicationSession
- StopApplicationSession
- GetDevices
- GetDevice
- SetAudioDeviceId
- SetVideoDeviceId
- On

Device Service

- GetCall
- GetCalls
- MakeCall
- JoinCall
- MonitorStart
- MonitorStop
- DirectedPickupCall
- SetForwarding
- GetForwarding
- SetDoNotDisturb
- GetDoNotDisturb
- SnapshotDevice
- SnapshotCall
- GetPresenceState

Call Services

- AnswerCall
- ClearConnection
- UpdateCall
- SingleStepTransferCall
- HoldCall
- RetrieveCall
- GenerateDigits
- DeflectCall
- DirectedPickupCall
- TransferCall
- ConferenceCall
- PlayMessage
- RecordMessage
- Stop

Supported Events:

Agent Events

- ApplicationSessionStartedEvent
- ApplicationSessionTerminatedEvent
- LocalStreamEvent
- RemoteStreamEvent

Device Events

- PresenceStateEvent

Call Events

- ServiceInitiatedEvent
- FailedEvent
- ConnectionClearedEvent
- OriginatedEvent
- DeliveredEvent
- DivertedEvent
- EstablishedEvent
- HeldEvent
- RetrievedEvent
- TransferredEvent
- DtmfDetectedEvent
- StopEvent

5.2 ANCTI AGENT METHODS & EVENTS

OVERVIEW ANCTI AGENT METHODS & EVENTS

- The agent is the main entry-point to the anCTI API.
- It handles the signaling to the anSwitch V7 and allows to register event-handlers.
  - Agent Services
    - StartApplicationSession
    - StopApplicationSession
    - GetDevices
    - GetDevice
    - SetAudioDeviceId
    - SetVideoDeviceId
    - On
  - Agent Events
    - ApplicationSessionStartedEvent
    - ApplicationSessionTerminatedEvent
    - LocalStreamEvent
    - RemoteStreamEvent

AGENT CREATION & METHODS

- Agent Constructor

  `AnCti.Agent()`

  - Creates a new instance of an agent
  - Use the AnCti namespace to access it.

- Agent Methods

  `startApplicationSession(options)`

  - This method starts an application session that represents the connection to the AS7 server.

  `stopApplicationSession()`

  - This method stops the application session and closes the link to the server.

  `getDevices()`

  - This method returns an array of all devices that were created on this agent

  `getDevice(options)`

  - This method returns the device with the given deviceID.
  - If no device with the given deviceID exists, a new one is created

  `getSessionDevice()`

  - This method returns the device that was used to start the application-session.

AGENT METHODS

- Agent Methods

`setAudioDeviceId(deviceID)`

- This defines the microphone that will be used for WebRTC audio-calls.

`setVideoDeviceId(deviceId)`

- This defines the camera that will be used for WebRTC video-calls.

`setAudioDeviceId(deviceID)`

- This defines the microphone that will be used for WebRTC audio-calls.
- If no value is specified, the browser will use the default device.

`setVideoDeviceId(deviceId)`

- This defines the camera that will be used for WebRTC video-calls.
- If no value is specified, the browser will use the default device.

AGENT EVENTS

Agent Event Method

`on(event, handler)`

- This add an event-handler for the given event.
- Currently the following events are supported for agents:
  - applicationsessionstarted
  - applicationsessionterminated
  - localstream
  - remotestream

Agent Events

`ApplicationSessionStartedEvent`

- This event is emitted if the application-session is successfully established.
- It contains additional information about the authenticated users.

`ApplicationSessionTerminatedEvent`

- This event is emitted if the application-session is terminated.

`LocalStreamEvent`

- This event is emitted if the local stream of a call changes.
- It contains a reference to the MediaStream.

`RemoteStreamEvent`

- This event is emitted if the remote stream of a call changes.
- The usage is the same as for `LocalStreamEvent`

5.3 ANCTI DEVICE METHODS & EVENTS

OVERVIEW ANCTI DEVICE METHODS & EVENTS

- The device represents either a physical device like a phone, or it represents a phone number of a PBX.

- Device Service
  - GetCall
  - GetCalls
  - MakeCall
  - JoinCall
  - MonitorStart
  - MonitorStop
  - DirectedPickupCall
  - SetForwarding
  - GetForwarding
  - SetDoNotDisturb
  - GetDoNotDisturb
  - SnapshotDevice
  - SnapshotCall
  - GetPresenceState

- Device Events
  - PresenceStateEvent

DEVICE METHODS

- Device Methods

`getCall( callID )`

- Returns the call with the given callID if it exists on this device.

`getCalls()`

- Returns an array of all calls that are currently bound to the device.

`monitorStart( [options] )`

- Starts monitoring the current device.
- The optional parameter allows to specify the type of monitoring.

`monitorStop()`

- Stops the monitoring of the current device.

`directedPickupCall( call )`

- Routes the specified call to the current device. If the destination supports the auto-answer feature then it will immediately accept the call without ringing in the first place.

`setForwarding( forwardingType , activateForward , forwardDN )`

- Configures a forwarding option on the current device.
- The following forwarding-types are supported:
  - forwardImmediate
  - forwardBusy
  - forwardNoAns.

`getForwarding()`

- Retrieves configured forwarding for the current device.
- The method returns a promise that resolves to an array of forwarding configurations.

DEVICE METHODS

Device Methods

`makeCall( dest, [constraints] )`

- Initiate a call for a device to a destination.
- Supports media call characteristics: audio, video
- Supports subject of calls:
  - Barge In
    Both parties of the call can hear and talk to the supervisor.
  - Whisper
    Only the selected party can hear the supervisor.
  - Listen
    The supervisor listen to both parties. No party of the call can hear and talk to the supervisor.
  - Predictive call
    First a connection will be established to the B side then a free A side will be selected and connected

`joinCall( callID, [constraints] )`

- Joins an existing connection
- Supports participation types:
  - Barge In, Whisper, Listen

`makeCall continued`

- It is possible to define a different caller id for an outgoing call. This is a proprietary feature and not specified by CSTA.
  - For using this feature, it is mandatory to define the OrgUnit attribute "clipSignaling" in the desired OrgUnit e.g.: PBX, department that contains all agents, etc.

OrgUnit Attribute Name	Value	Remark
clipSignaling	[any | pbx]	any: Every number can be specified and will be used as preferred public number (Like "special arrangement" in anSwitch V6) pbx: Every public number of the PBX can be used.

DEVICE METHODS

- Device Methods

`setDoNotDisturb( doNotDisturbOn )`

- Configures the do-not-disturb option on the current device.

`getDoNotDisturb()`

- Retrieves current state of the do-not-disturb of a device.
- The method returns a promise that resolves to an array of forwarding configurations.

`SnapshotDevice( termID )`

- This service is used to detect ongoing calls on a device.

`SnapshotCall ( callID, termID )`

- Once an application detected an ongoing call on a device, it can use this service to get information about the other devices attached to the call.

DEVICE EVENTS

- Device Event Methods

  `GetPresenceState ( termID )`
  - This service is used to detect if there is at least one active (SIP) registration for the given device.

- Device Events

  `PresenceStateEvent`
  - Once an application uses GetPresenceState it is automatically subscribed to get `PresenceStateEvents`.
  - These event are emitted as soon as the server detects a change:
    - If a phone (de-)registers via SIP the event is emitted immediately.
    - If a phone fails to re-register this is not detected immediately. Rather the system polls the status and emits the event at the next poll-period.

5.4 ANCTI CALL METHODS & EVENTS

OVERVIEW ANCTI CALL METHODS & EVENTS

- The Call class represents the appearance of a call on a device

- Call Service
  - AnswerCall
  - ClearConnection
  - UpdateCall
  - SingleStepTransferCall
  - HoldCall
  - RetrieveCall
  - GenerateDigits
  - DeflectCall
  - DirectedPickupCall
  - TransferCall
  - ConferenceCall
  - PlayMessage
  - RecordMessage
  - Stop

- Call Events
  - ServiceInitiatedEvent
  - FailedEvent
  - ConnectionClearedEvent
  - OriginatedEvent
  - DeliveredEvent
  - DivertedEvent
  - EstablishedEvent
  - HeldEvent
  - RetrievedEvent
  - TransferredEvent
  - DtmfDetectedEvent
  - StopEvent

CALL METHODS

- Call Methods

`answerCall( [constraints] )`

- Answers the incoming call.
- The optional constraints allows to define if only audio, or also video shall be answer..

`clearConnection()`

- Terminates the call.

`updateCall( [constraints] )`

- Similar to answerCall but allows to change the media-constraints of an ongoing call.
- This method is not defined by the CSTA specs. It can be seen as a generalized version of hold/retrieve that allows to control audio and video-streams individually.

`singleStepTransferCall( dest )`

- Performs a single-step-transfer of the call to the new destination.

`holdCall()`

- Puts the call on hold.

`retrieveCall()`

- Retrieves a previously held call.

`generateDigits( digits )`

- Instructs the browser to generate RFC2833 inband DTMF events.

CALL METHODS

Call Methods

`deflectCall( dest )`

- Deflects the call to the new destination.

`directedPickupCall( number )`

- Pickups a call from another device.

`transferCall( otherCall )`

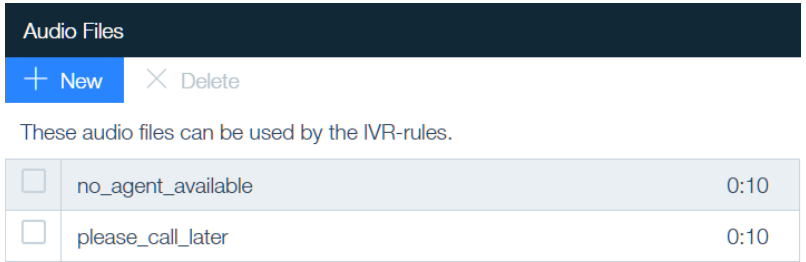
- Transfers the current call to another one.

`conferenceCall( otherCall )`

- Conferences the current call with the other one.
- If a device contains connections on both calls one connection will be dropped.

`playMessage( messageToBePlayed )`

- Plays the specified audio file.
- For ACD or IVR the audio files are referenced by their name.



`recordMessage()`

- Starts recording the call.

`stop( messageToBeStopped )`

- Used to stop playing a message.
- Used to stop recording a message.

CALL EVENTS

Call Event Method

`on("call", handler)`

- This add an event-handler for the given call event.
- Currently the anSwitch V7 sends the following supported events:
  - ServiceInitiatedEvent
  - FailedEvent
  - ConnectionClearedEvent
  - OriginatedEvent
  - DeliveredEvent
  - DivertedEvent
  - EstablishedEvent
  - HeldEvent
  - RetrievedEvent
  - TransferedEvent
  - DtmfDetectedEvent
  - StopEvent

Call Events

`CallEvent`

- These events are emitted when the anSwitch V7 sends a call-related CSTA event as defined in ECMA-269 Standard.

5.5 ANCTI PHONE METHODS & EVENTS

OVERVIEW ANCTI PHONE METHODS & EVENTS

- The phone offers an easy way to create a phone-like application.
  - In contrast of the Device instance a Phone provides a higher abstraction.

  - Phone Service
    - setPrimaryDevice
    - setToneUrl
    - setCamera
    - setMicrophone
    - setSpeaker
    - setRinger
    - setRemoteVideoElement
    - setLocalVideoElement

PHONE METHODS

Phone Methods

`setPrimaryDevice( deviceId )`

- Used to select the device for which the phone shall control the calls.

`setToneUrl( tone , url )`

- This method configures an url to an external audio-file to be used when the tone shall be placed.

`setCamera( deviceId )`
`setMicrophone( deviceId )`
`setSpeaker( deviceId )`
`setRinger( deviceId )`

- Defines which camera etc. to use.

`setRemoteVideoElement( element )`

- If configured the phone will automatically bind the remote-video-stream to the reference HTMLVideoElement.

`setLocalVideoElement( element )`

- If configured the phone will automatically bind the local-video-stream to the reference HTMLVideoElement.

5.6 EXAMPLE OF A VERY SIMPLE WEBPHONE WITH THE ANCTI LIBRARY

EXAMPLE OF A VERY SIMPLE WEBPHONE IMPLEMENTATION

- Example of a Very Simple WebPhone using the ancti.js JavaScript library
  - The example shows the relevant concepts of the anCTI library.
  - Use the user account's username and password (the same as for a SIP phone) for authentication in `StartApplicationSession()`.
  - In this example a call toward destination number 23 is started immediately when there is no active call.
    Your application should collect the destination number first and then the call should be started with `makeCall()`.
- Access all anCTI JavaScript examples of your anSwitch V7 :

  https://PROVIDER_DOMAIN/cti/examples/README.html

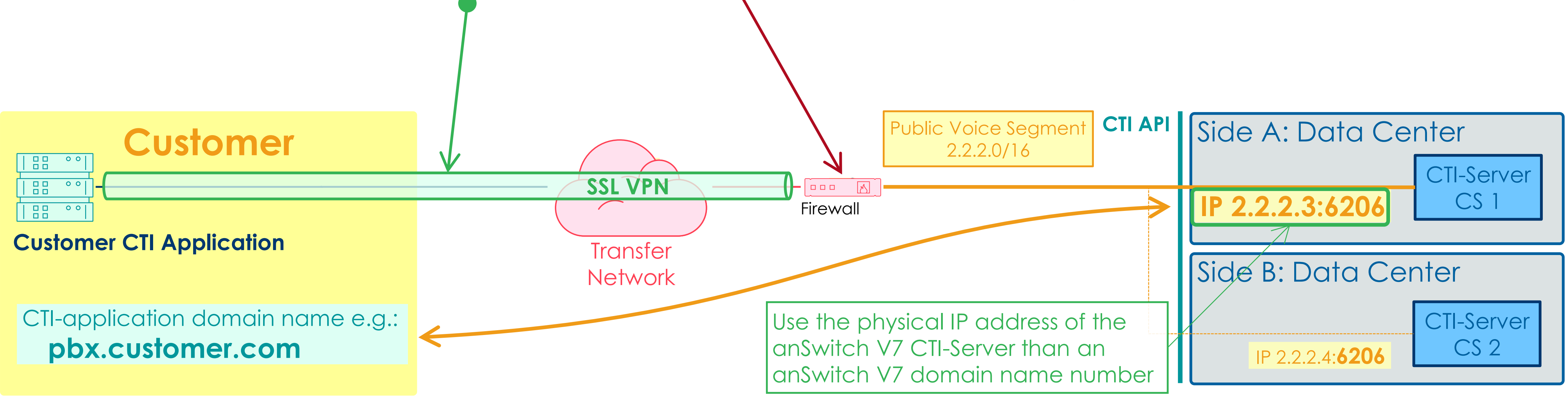
  or here (but not always online):
  https://as7poc.aarenet.com/cti/examples/README.html

```
<html lang="en">
 <head>
  <title>Video-Phone</title>
  <script src="https://provider.com/cti/ancti/ancti.min.js"></script>
 </head>
 <body>
  <video id="localView"  playsInline autoPlay muted></video>
  <video id="remoteView" playsInline autoPlay></video>
  <script>
    var agent = AnCti.newAgent();
    var myDevice = undefined;

    await agent.startApplicationSession({username: "alice@example.com",password: "secret",
                                         sessionDevice: "sip:280@term.572"})
    const myDevice = agent.getSessionDevice();

    // tell server that we want to use WebRTC (error handling omitted)
    myDevice.monitorStart({ rtc:true });

    // if WebRTC creates a media-stream we bind it to the corresponding video element
    agent.on('remotestream',(event) => document.getElementById('remoteView').srcObject = event.stream);
    agent.on('localstream', (event) => document.getElementById('localView').srcObject = event.stream);

    document.onclick = () => {
        let call = myDevice.calls[0];
        if (!call) {
            // click without an active call -> start a video call to number 23
            myDevice.makeCall("23", { autoOriginate: "doNotPrompt", audio: true, video: true });
        } else if (call.localConnectionInfo=="alerting") {
            // click while we have an alerting call -> accept it
            call.answerCall({ audio:true, video: true});
        } else {
            // otherwise, we release the call
            call.clearConnection();
        }
    }
  </script>
 </body>
</html>
```

6 CTI API RELATED IT & SECURITY CONSIDERATIONS

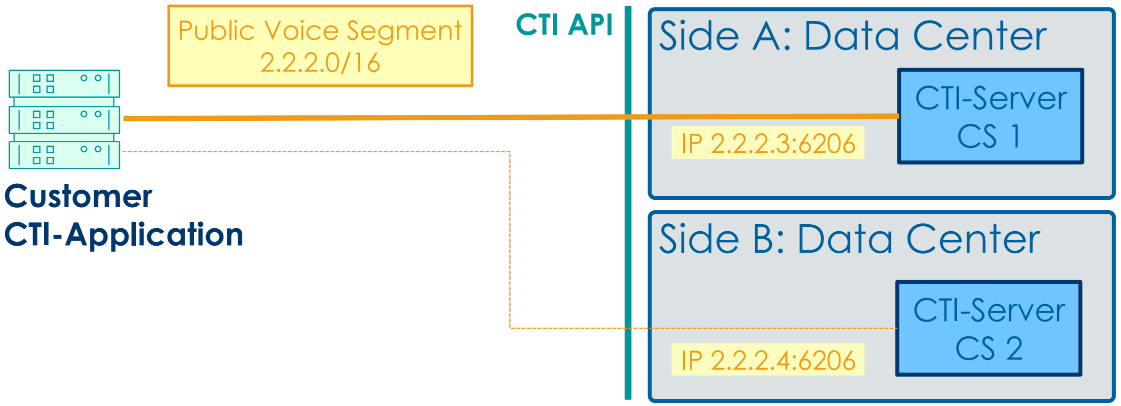
IP NETWORK & SECURITY FOR PURE CSTA CTI API

- The IP routing between the CTI-application and the Public VoIP segment of the anSwitch V7 must be end-to-end:
  - Configure appropriate firewall policies for all customer CTI-applications:
    - Pure CSTA policy:
      Customer domains ←→ CTI-Server IP address, TCP port: 6206
- As TCP is used, it is up to the customer or provider to setup the appropriate security concept on IT level, e.g. a SSL or IPSec VPN.



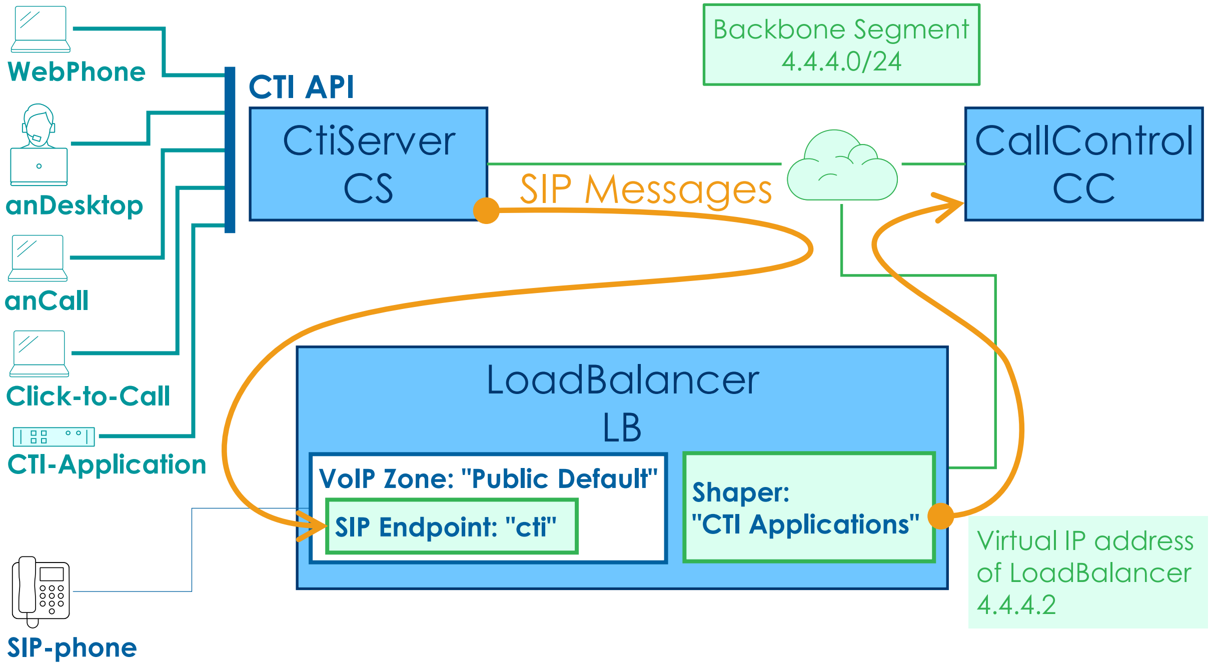
HIGH AVAILABILITY FOR THE PURE CSTA CTI API

- Considerations of the high availability for pure CSTA CTI API of the CTI-Server
  - During normal running condition, a CTI-application just establishes a TCP connection to the primary anSwitch V7 CTI-Server e.g. CS 1.
  - If the connection is broken and cannot be reestablished, the CTI-application should switch over and use the secondary anSwitch V7 CTI-Server as fall back.



TRAFFIC SHAPER AGAINST OVERLOAD FROM CTI APPLICATIONS

- Apply a Traffic Shaper against overload from applications that use the CTI APIs.
  - The SIP Endpoint "cti" must be created.
    (Doc-ID: training_as7_601_voip_zones)
  - Configure a shaper, e.g. "CTI Applications" and assign it to the default public VoIP Zone, e.g. "Public Default".
    (Doc-ID: training_as7_713_sys_security)



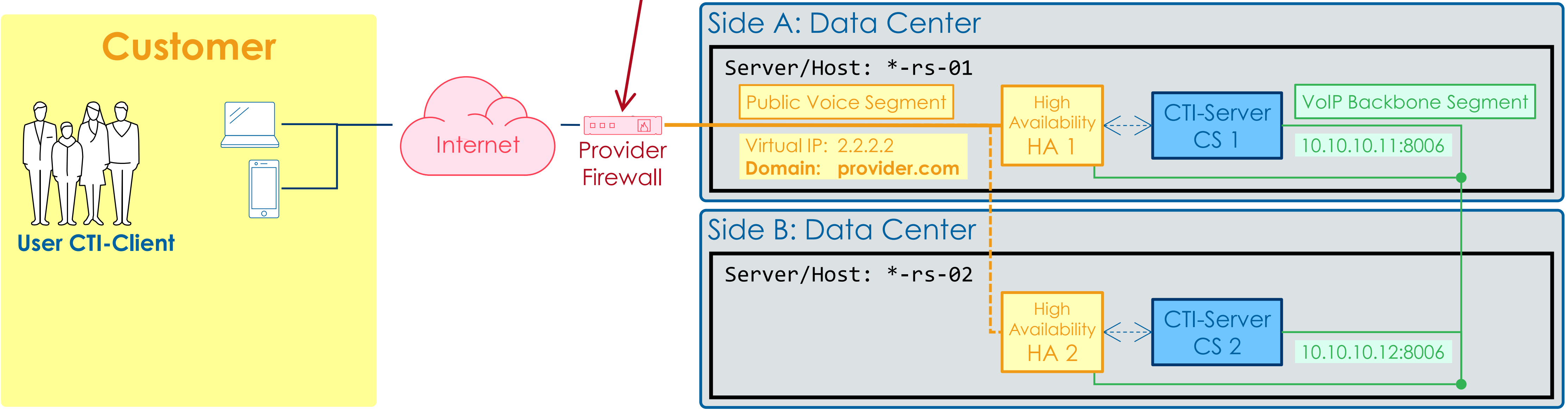
Settings	
Name	CTI Applications
Subnet	4.4.4.0/24
VoIP Zone	Public Default

Network Shaping	
Invites Network	40
Registers Network	40
Subscriptions Network	40
Options Network	40
Other Methods Network	40
Responses Network	40

Device Shaping	
Invites Device	0
Registers Device	0
Subscriptions Device	0
Options Device	0
Other Methods Device	0
Responses Device	0

IP ROUTING & SECURITY FOR CTI API USING THE ANCTI LIBRARY

- The IP routing between the customer's CTI-clients and the Public VoIP segment of the anSwitch is end-to-end:
  - anCTI library CSTA policy:
    Internet located CTI-Clients ←→ CTI-Server IP-address, HTTPS, TCP port: 443
- Security is ensured by the HTTPS protocol used and mandatory authentication at the start of an application session.



SECURITY ASPECTS FOR MONITORING CTI ENTITIES

- Security aspects:
  - An explicit permission is required to monitor a certain terminal or PBX Extension.
  - As an Operator can grant monitoring permissions to any user, it would be possible that the Operator allows to monitor a terminal or PBX Extension of a different PBX.
    - ➔ Therefore, the CTI-Server checks implicitly that the user is in the same PBX as the monitored entity.
    - ➔ This ensures that neither the user nor the PBX Administrator can "breakout" of their PBX.

7 CTI API ANSWITCH V7 SYSTEM CONFIGURATIONS

THE CTI API DEFAULT SIP-ENDPOINT

- To do calls from a browser or a Web Phone using the anCTI JavaScript library WebRTC is used.
  - The CTI-Server acts as a gateway for the CallControl that maps WebRTC to SIP.
  - The CtiServer component needs a SIP-Endpoint for communicating with the CallControl component.
    - The system default SIP-Endpoint for CTI is named: "cti"
  - Even if the CtiServer itself uses private IP addresses, it must use a SIP-Endpoint that uses public IP-addresses.

- For setting up a different named SIP-Endpoint for the CTI-Server Aarenet system engineers are required.
  - The /etc/aareswitch/system.yaml must be modified as follows, e.g.: "Default Public"
  - The SIP-Endpoint name is case-sensitive!

```
components:
 cs*:
#   SIP-Endpoint the CtiServer and CallControl must use for exchanging SIP messages.
    peerSipEndpoint:          "cti"
```

HIGH AVAILABILITY HA FOR THE CTI API

- For using the anCTI Library of the CTI API, it needs the HA service of the anSwitch V7.
- The HA service provides the needed certificate for HTTPS for the CTI-clients using the anCTI library.
- For setting up the HA service Aarenet system engineers are required.

Note	The HA service is configured that it points primarily to the CTI-Server CS 1 component. The CTI-Server CS 2 component stays hot standby. If CS 1 fails, CS 2 takes over automatically, when CS 1 is available again, the system switches back to CS 1. When switching between CS 1 and CS 2 and vice versa, information of managed calls may be lost.

MAPPING SIP HEADER USER-TO-USER INTO CSTA MESSAGES

1. Enable the mapping of SIP Header User-to-User transparently into CSTA messages if needed:
   a. Modify the system.yaml file on all server/host:

```
csta:
  mapUserInfo:
    "User-to-User.CallGUID":  "userData/CallGUID"
```

   b. Restart all server/host

2. As User-to-User may transport sensitive data give permission to access this information.
   - These permissions can be granted like other terminal-configurations either as terminal- or OU-attribute ('terminal.<type>.<name>')

Terminal / OU Attribute	Value	Description
evalUserInfo	User-to-User	Accept the SIP header User-To-User.
sendUserInfo	User-to-User	It is allowed to map the SIP header User-To-User.

MAPPING SIP HEADER USER-TO-USER INTO CSTA MESSAGES

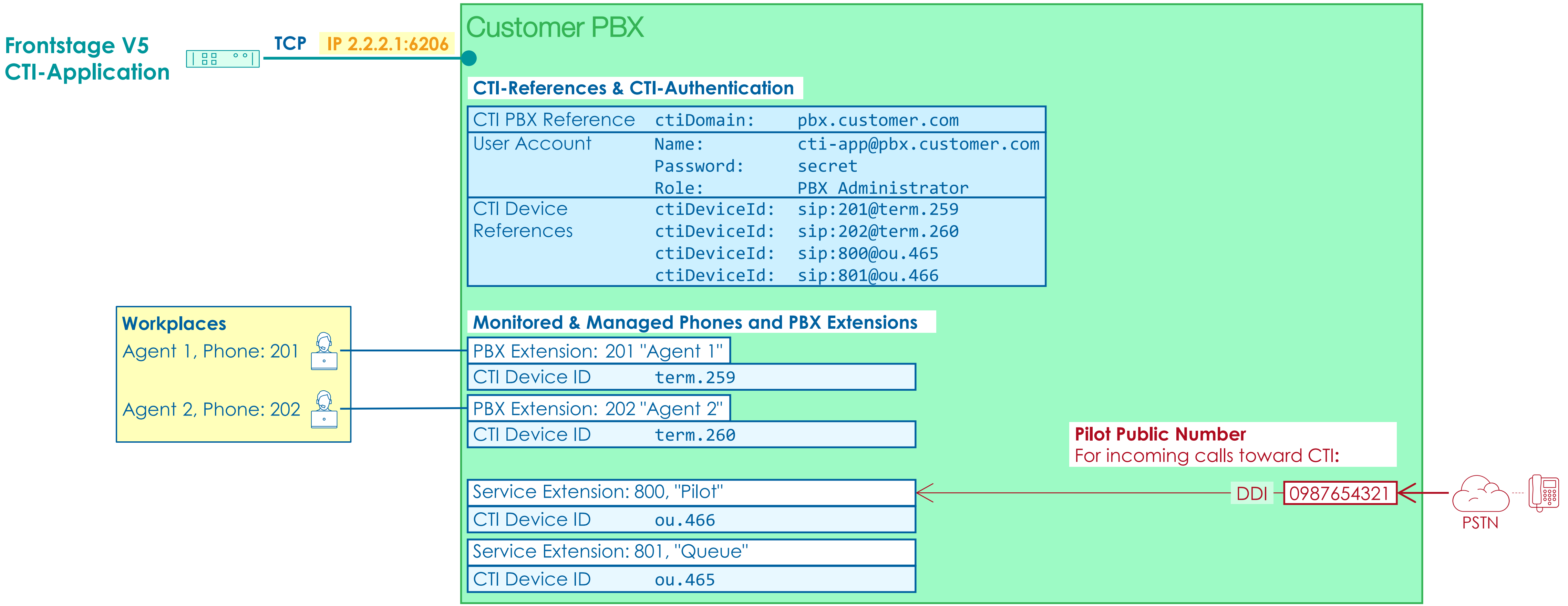
- The result of these configurations are that the value of the "CallGUID" parameter in the User-to-User SIP header is mapped into the UserData-element of the Delivered- and Established events.

```
<DeliveredEvent xmlns="http://www.ecma-international.org/standards/ecma-323/csta/ed4">
  <monitorCrossRefID>413</monitorCrossRefID>
  <connection>
    <callID>10004245</callID>
   <deviceID>2653@term.3525</deviceID>
  </connection>
  ...
  <userData>
    <CallGUID>xxxxxxx</CallGUID>
  </userData>
</DeliveredEvent>
```

8 CONFIGURATIONS FOR FRONTSTAGE V5

OVERVIEW INTERWORKING FRONTSTAGE V5 WITH THE PBX

Setup for Frontstage V5



CONFIGURATION PARAMETER FOR FRONTSTAGE V5

- Provide the following information for the configuration of Frontstage V5:

Category	Parameter Name	Examples Values	Remark
CTI PBX Reference	PBX-reference	pbx.customer.com	The CTI PBX reference is coupled with the Frontstage V5 license.
CTI-Application Authentication	Username	cti-app@pbx.customer.com	Credentials from the CTI-application anSwitch V7 user account.
	Password	secret	
Public anSwitch V7 CTI	IP address and Port	2.2.2.1:6206	For the IP check with the telephony provider.
	TCP session expiration time	20'000ms	
Public Number	Public number of "Pilot"	0987654321	DDI to "Pilot" 800
Internal Numbers	Internal number length	3 digits	
	Workplaces	201, 202	List of the internal numbers of the managed workplaces
	Service Extension "Pilot"	800	
	Service Extension "Queue"	801	
CTI Device References	CTI device ID "Pilot"	ou.465	More workplaces can be added. The ID is generated when a phone is associated to the PBX Extension.
	CTI device ID "Queue"	ou.466	
	CTI device ID "Workplace 201"	term.259	
	CTI device ID "Workplace 202"	term.260	

- The resulting PBX connection CSV string → example values:

```
2.2.2.1,6206,20000,3,cti-app@pbx.customer.com,secret,pbx.customer.com
```

- The resulting CTI device references → example values:

```
sip:800@ou.465
sip:801@ou.466
sip:201@term.259
sip:202@term.260
```

CONFIGURATION FOR FRONTSTAGE V5 APPLICATIONS

Configuration example:

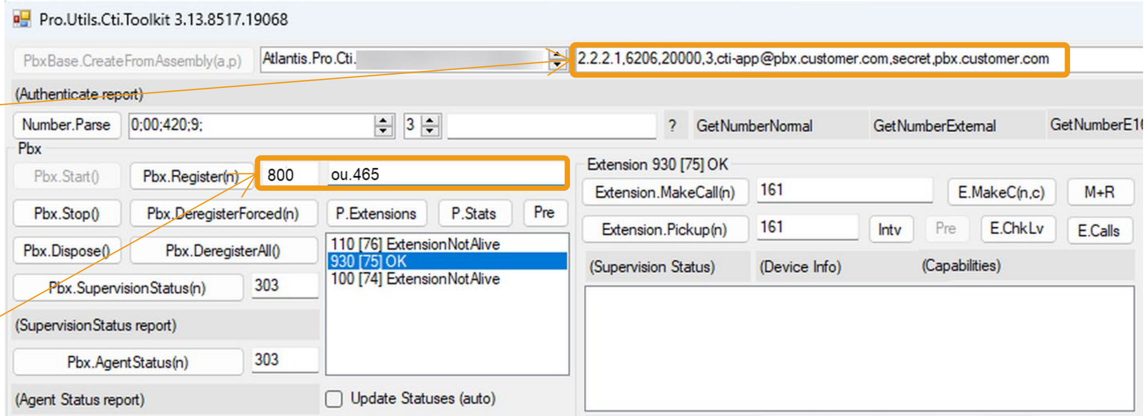
- Configure the PBX connection CSV string:

  2.2.2.1,6206,20000,3,cti-app@pbx.customer.com,secret,pbx.customer.com

- Configure the CTI device references:

  sip:800@ou.465
  sip:801@ou.466
  sip:201@term.259
  sip:202@term.260

Development tool "Pro.Utils.Cti.Toolkit":



Frontstage V5 GUI:

LAST PAGE

Date	Doc-ID	Description	Changes
25.4.2023	training_as7_704_api_callcontrol_cti_integration_e23	Remove old style of terminal identification	Chapter 2.1 overdone
14.7.2023	training_as7_704_api_callcontrol_cti_integration_e30	Overdone chapter structure and their contents	Chapters overdone 2, 3, 4, 5; New chapter: 4.8, 8; Page: 101
23.10.2023	training_as7_704_api_callcontrol_cti_integration_e31	New default SIP Endpoint for CTI	Page: 98, 102
31.1.2025	training_as7_704_api_callcontrol_cti_integration_e32	V7.15: Introduce globallyUniqueCallLinkageID Use phone user/password V7.18: New anCTI "Phone" Class	New chapter: 4.9 Page: 81, 99 New chapter: 5.6
28.4.2025	training_as7_704_api_callcontrol_cti_integration_e321	Correct title: Get the CDR of a Call via CSTA & REST API	Page: 73, 74